SecuTrack

Documentation technique

Application de gestion de tickets de sécurité : architecture, backend Spring Boot, sécurité JWT, frontend Angular, tests, Docker, GitHub Actions et TeamCity. Chaque partie est expliquée avec le code réel du projet.

Java 21 · Spring Boot 3.5 · Spring Security / JWT · JPA / Hibernate · PostgreSQL · Angular 22 · Angular Material · Vitest · Docker · GitHub Actions · TeamCity

Table des matières

1. Présentation 3
2. Démarrer le projet 5
3. Architecture 7
4. Backend : démarrage et configuration 9
5. Modèle de données (JPA) 12
6. Repositories et requêtes 15
7. Services et règles métier 17
8. Contrôleurs et API REST 20
9. Validation et gestion des erreurs 23
10. Sécurité : Spring Security et JWT 26
11. Tests du backend 30
12. Frontend Angular : les fondations 32
13. Frontend Angular : les pages 37
14. Docker 40
15. CI/CD avec GitHub Actions 43
16. TeamCity 45
17. Contribuer : commits et pull requests 47
18. Analyse des besoins et Agile 50
19. Tester l'API et explorer la base 52
20. Les outils d'IA dans le développement 53
21. Vigie technologique et exploitation 54
22. Couverture du poste et plan de mise à niveau 55
23. Aide-mémoire des annotations 56
24. Dépannage 58

1. Présentation

SecuTrack est une application interne où les employés déclarent des **tickets de sécurité** (incidents, vulnérabilités, demandes d'accès) et où les administrateurs les trient, les assignent et les résolvent. Elle reprend la stack du poste Développeur I - Full Stack : Java et Spring Boot pour le backend, Angular pour le frontend, Docker et des pipelines CI/CD avec GitHub Actions et TeamCity.

Ce document accompagne le code. Le code lui-même est commenté : chaque annotation (`@Entity`, `@Service`, `@Transactional`...) et chaque concept Angular (`signal`, `inject`, `input`...) est expliqué à l'endroit où il est utilisé. Ici, tu trouves la vue d'ensemble : comment les pièces s'assemblent et pourquoi.

La stack technique

Couche	Technologies	Où dans le projet
Backend	Java 21, Spring Boot 3.5, Spring Web, Spring Data JPA, Hibernate, Bean Validation	`backend/`
Sécurité	Spring Security, JWT HS256 (resource server), rôles USER / ADMIN, BCrypt	`backend/.../config`, `auth`
API	REST, ProblemDetail (RFC 9457), pagination, OpenAPI / Swagger UI	`*Controller.java`
Base de données	H2 en développement, PostgreSQL 16 avec Docker	`application*.yml`
Frontend	Angular 22 standalone, signals, Reactive Forms, Angular Material	`frontend/`
Tests	JUnit 5, Mockito, AssertJ, MockMvc · Vitest	`src/test`, `*.spec.ts`
Conteneurs	Dockerfiles multi-stage, docker compose, nginx	`Dockerfile`, `docker-compose.yml`
CI/CD	GitHub Actions + GHCR, TeamCity (Kotlin DSL), Dependabot	`.github/`, `.teamcity/`

Les fonctionnalités

Fonctionnalité	Backend	Frontend
Inscription et connexion avec JWT	`AuthController`, `AuthService`, `JwtService`	Pages Connexion et Inscription
Rôles USER et ADMIN	`SecuriteConfig`, `@PreAuthorize`, `TicketService`	`adminGuard`, boutons conditionnels
CRUD des tickets	`TicketController`, `TicketService`	Tableau de bord, détail, formulaire
Recherche, filtres, pagination, tri	`TicketSpecifications`, `Pageable`	Filtres, `mat-paginator`, `matSort`
Statistiques	`GET /api/tickets/statistiques`	Cartes du tableau de bord
Assignation	`PATCH /api/tickets/{id}/assignation`	Liste déroulante (ADMIN)
Administration des rôles	`UtilisateurController`	Page Utilisateurs (ADMIN)

Fonctionnalité	Backend	Frontend
Validation et erreurs propres	Bean Validation, `GlobalExceptionHandler`	Messages par champ, notifications
Documentation de l'API	springdoc OpenAPI	`/swagger-ui.html`

Les comptes de démonstration

Au premier démarrage, si la base est vide, la classe `DonneesDemo` crée trois comptes et cinq tickets. Sur la page de connexion, les boutons « Admin » et « Utilisateur » remplissent le formulaire automatiquement.

Courriel	Mot de passe	Rôle
`admin@secutrack.local`	`Demo1234!`	ADMIN
`analyste@secutrack.local`	`Demo1234!`	USER
`dev@secutrack.local`	`Demo1234!`	USER

2. Démarrer le projet

Prérequis

Un JDK 21, Node.js 24 et Docker Desktop. Maven n'est pas nécessaire : le script `backend/mvnw` (Maven Wrapper) télécharge automatiquement la bonne version de Maven au premier lancement.

Option 1 : tout lancer avec Docker

```
docker compose up --build
```

Docker construit les deux images, démarre PostgreSQL, puis l'API, puis nginx. Ouvre ensuite http://localhost:8081. L'API reste aussi accessible directement sur http://localhost:8080.

Option 2 : mode développement (rechargement à chaud)

```
# Terminal 1 : API sur http://localhost:8080 (H2 en mémoire + données de démo)
cd backend
./mvnw spring-boot:run

# Terminal 2 : Angular sur http://localhost:4200
cd frontend
npm install
npm start
```

En développement, `frontend/proxy.conf.json` relaie les appels `/api` d'Angular (port 4200) vers l'API (port 8080). Pour le navigateur, tout vient du même serveur : pas de problème de CORS.

Lancer les tests

```
cd backend  && ./mvnw verify                 # tests unitaires + tests d'intégration
cd frontend && npx ng test --watch=false     # tests Vitest
```

Explorer l'API avec Swagger UI

Ouvre http://localhost:8080/swagger-ui.html (ou :8081 avec Docker). Appelle `POST /api/auth/login` avec un compte de démonstration, copie la valeur `token` de la réponse, clique sur **Authorize** et colle-la : tous les autres endpoints deviennent testables.

> **Bonne pratique** — Teste aussi les erreurs depuis Swagger : un titre vide (400), un ticket inexistant (404), une suppression avec le compte analyste (403), une modification de ticket fermé (409). C'est la meilleure façon de comprendre le `GlobalExceptionHandler`.

Les documents du projet

Fichier	Contenu
`README.md`	Démarrage rapide, architecture, configuration
`CONTRIBUTING.md`	Branches, commits, pull requests, revue de code
`docs/SecuTrack_Documentation.pdf`	Ce document

Fichier	Contenu
`docs/EXERCICES.md`	Exercices Git, TeamCity, backlog et plan de mise à niveau
`docs/ANALYSE-ET-AGILE.md`	User stories, critères d'acceptation, sprints
`docs/TESTER-L-API.md` + `docs/postman/`	Swagger, collection Postman, curl
`docs/BASE-DE-DONNEES.md` + `docs/sql/`	Connexion avec un client SQL, requêtes utiles
`docs/IA-DEVELOPPEMENT.md`	Outils d'IA : règles et exercices
`docs/VIGIE.md`	Versions, fins de support, note de vigie
`docs/EXPLOITATION.md`	Déploiement, surveillance, sauvegarde, incidents
`docs/GUIDE-UTILISATEUR.md`	Guide pour les utilisateurs finaux

3. Architecture

Organisation du dépôt

```
secutrack/
├── backend/                          API Spring Boot
│   ├── src/main/java/com/secutrack/
│   │   ├── SecutrackApplication      point d'entrée (main)
│   │   ├── config/                   sécurité, JWT, CORS, OpenAPI, propriétés
│   │   ├── auth/                     inscription, connexion, génération des JWT
│   │   ├── utilisateur/              entité Utilisateur, rôles, administration
│   │   ├── ticket/                   entité Ticket, recherche, règles métier, API
│   │   ├── common/                   exceptions métier + gestion globale des erreurs
│   │   └── demo/                     données de démonstration
│   ├── src/main/resources/           application.yml, application-docker.yml
│   ├── src/test/java/                tests JUnit
│   ├── Dockerfile, pom.xml, mvnw
├── frontend/                         application Angular
│   ├── src/app/core/                 session, intercepteur JWT, guards, erreurs, notifications
│   ├── src/app/tickets/              modèles TypeScript + services HTTP
│   ├── src/app/pages/                une page = un dossier (TS + HTML + SCSS + tests)
│   ├── src/app/shared/               composants réutilisables
│   ├── Dockerfile, nginx.conf, proxy.conf.json
├── docker-compose.yml                PostgreSQL + backend + frontend
├── .github/                          workflow CI/CD, Dependabot, modèle de PR
├── .teamcity/settings.kts            pipeline TeamCity
├── teamcity/                         serveur + agent TeamCity locaux
└── docs/                             cette documentation + exercices
```

Le backend est organisé **par fonctionnalité** : tout ce qui concerne les tickets est dans `ticket/` (entité, repository, service, contrôleur, DTO). Les classes qui changent ensemble restent ensemble, ce qui rend les pull requests plus lisibles. Une autre convention courante est le découpage par couche (`controller/`, `service/`…) : adopte celle de ton équipe.

Les couches du backend

```
Client (Angular, curl, Swagger)
   │  HTTP + JSON
   ▼
Filtres Spring Security      vérifie le JWT (401) puis les règles d'URL (403)
   ▼
Controller  @RestController  reçoit, valide (@Valid), répond           → DTO
   ▼
Service     @Service         règles métier, @Transactional              DTO ↔ entité (mapper)
   ▼
Repository  JpaRepository    accès aux données, requêtes               → entités
   ▼
Hibernate → SQL → H2 / PostgreSQL
```

Chaque couche ne parle qu'à celle du dessous. Le contrôleur ne touche jamais au repository ; le repository ne connaît pas les DTO ; les règles métier ne sont jamais dans le contrôleur.

Le parcours d'une requête authentifiée

Exemple : un analyste crée un ticket depuis Angular.

Étape	Où	Ce qui se passe
1	`authInterceptor` (Angular)	Ajoute `Authorization: Bearer <jeton>` à la requête `POST /api/tickets`.

Étape	Où	Ce qui se passe
2	nginx	Relaie `/api/...` vers le conteneur backend (port 8080).
3	`BearerTokenAuthenticationFilter`	Vérifie la signature et l'expiration du JWT avec le `JwtDecoder`. Invalide : 401.
4	`jwtAuthenticationConverter`	Transforme la claim `roles: ["USER"]` en autorité `ROLE_USER`.
5	Règles d'URL (`SecuriteConfig`)	`/api/tickets` exige d'être connecté : accepté.
6	Jackson + `@Valid`	Convertit le JSON en `TicketRequest` et vérifie `@NotBlank`, `@Size`… Échec : 400.
7	`TicketController.creer`	Récupère le courriel depuis le JWT (`jwt.getSubject()`) et appelle le service.
8	`TicketService.creer`	Transaction ouverte, charge le créateur, construit le `Ticket` au statut OUVERT.
9	Hibernate	`@PrePersist` remplit les dates, puis INSERT ; l'id est généré par la base.
10	Commit + réponse	`TicketMapper` produit le DTO ; le contrôleur renvoie 201 et le header `Location`.

4. Backend : démarrage et configuration

Le point d'entrée

backend/src/main/java/com/secutrack/SecutrackApplication.java

```
/**
 * Point d'entrée de l'application : c'est la classe qu'on lance (méthode main).
 *
 * IMPORTANT : cette classe doit rester dans le package racine "com.secutrack".
 * Spring ne cherche les classes annotées (@Service, @RestController...) que dans
 * ce package et ses sous-packages (config, auth, ticket...).
 */
// @SpringBootApplication regroupe 3 annotations :
//   - @Configuration            : cette classe peut déclarer des beans (objets gérés par Spring)
//   - @EnableAutoConfiguration  : Spring Boot configure tout seul ce qu'il trouve dans le pom
//                                 (base de données, serveur web Tomcat, Jackson pour le JSON...)
//   - @ComponentScan            : Spring parcourt les packages pour trouver nos classes annotées
@SpringBootApplication
// @ConfigurationPropertiesScan : enregistre automatiquement les classes annotées
// @ConfigurationProperties (ici SecutrackProperties, qui lit le bloc "secutrack:" du YAML)
@ConfigurationPropertiesScan
public class SecutrackApplication {

    public static void main(String[] args) {
        // Démarre Spring : crée tous les beans, puis lance le serveur Tomcat intégré sur le port 8080
        SpringApplication.run(SecutrackApplication.class, args);
    }
}
```

Le pom.xml et les starters

Le parent `spring-boot-starter-parent` fixe des versions compatibles pour toutes les dépendances Spring : c'est pour ça qu'elles n'ont pas de `<version>`. Chaque **starter** regroupe ce qu'il faut pour un besoin.

Dépendance	Rôle dans SecuTrack
`spring-boot-starter-web`	API REST, Tomcat intégré, conversion JSON (Jackson)
`spring-boot-starter-data-jpa`	Entités, Hibernate, repositories Spring Data
`spring-boot-starter-validation`	`@NotBlank`, `@Size`, `@Email` sur les DTO
`spring-boot-starter-security`	Chaîne de filtres, rôles, BCrypt
`spring-boot-starter-oauth2-resource-server`	Vérification des JWT + bibliothèque Nimbus pour les signer
`spring-boot-starter-actuator`	`/actuator/health` pour Docker et Kubernetes
`springdoc-openapi-starter-webmvc-ui`	Swagger UI généré depuis les contrôleurs
`h2` / `postgresql`	Pilotes JDBC (H2 en dev, PostgreSQL avec Docker)
`spring-boot-starter-test` / `spring-security-test`	JUnit 5, Mockito, AssertJ, MockMvc

application.yml

Toute la configuration qui change d'un environnement à l'autre est dans ce fichier. La syntaxe `${VARIABLE:defaut}` lit une variable d'environnement, avec une valeur par défaut pour le développement.

backend/src/main/resources/application.yml

```
# ====================================================================
# Configuration principale de l'API SecuTrack.
# ${VARIABLE:valeur} = lit la variable d'environnement VARIABLE, sinon utilise "valeur".
# Le profil "docker" (application-docker.yml) surcharge certaines valeurs.
# ====================================================================
spring:
  application:
    name: secutrack-api
  datasource:
    # Base H2 en mémoire pour le développement (perdue à chaque redémarrage).
    # MODE=PostgreSQL : H2 imite le comportement de PostgreSQL utilisé en production.
    url: jdbc:h2:mem:secutrack;DB_CLOSE_DELAY=-1;MODE=PostgreSQL
    username: sa
    password:
  jpa:
    hibernate:
      # update : Hibernate crée/modifie les tables d'après les entités (pratique en développement).
      # En production, on préfère "validate" + des migrations Flyway (voir ticket SEC-5).
      ddl-auto: update
    # false : pas de requêtes SQL cachées pendant la conversion en JSON ;
    # tout doit être chargé dans le service, à l'intérieur de la transaction.
    open-in-view: false
  threads:
    virtual:
      # Threads virtuels de Java 21 : Tomcat peut traiter beaucoup plus de requêtes simultanées
      enabled: true
  data:
    web:
      pageable:
        # Format JSON stable des pages : { "content": [...], "page": { size, number, totalElements, totalPages } }
        serialization-mode: via-dto

server:
  port: 8080

# Actuator : endpoints de supervision. On n'expose que health et info
# (jamais "*" : /actuator/env ou /heapdump peuvent révéler des secrets).
management:
  endpoints:
    web:
      exposure:
        include: health,info
  endpoint:
    health:
      probes:
        # Active /actuator/health/liveness et /readiness (sondes utilisées par Kubernetes)
        enabled: true

springdoc:
  swagger-ui:
    path: /swagger-ui.html

# Propriétés propres à SecuTrack, lues par la classe SecutrackProperties
secutrack:
  jwt:
    # Au moins 32 caractères (HS256). En production : variable d'environnement JWT_SECRET, jamais dans Git.
    secret: ${JWT_SECRET:secret-de-developpement-a-changer-absolument-0123456789}
    duree-minutes: 120
  demo:
    # Comptes admin@ / analyste@ / dev@secutrack.local et 5 tickets créés si la base est vide
    actif: ${DEMO_ACTIF:true}
    mot-de-passe: ${DEMO_MOT_DE_PASSE:Demo1234!}
```

```
  # Origines autorisées à appeler l'API depuis un navigateur (CORS) : le serveur de dev Angular
  cors-origines:
    - http://localhost:4200
```

Les profils : application-docker.yml

Quand `SPRING_PROFILES_ACTIVE=docker` (défini dans `docker-compose.yml`), Spring charge aussi ce fichier, dont les valeurs remplacent celles du fichier principal : on passe de H2 à PostgreSQL sans toucher au code.

`backend/src/main/resources/application-docker.yml`

```
# Profil "docker" : activé par SPRING_PROFILES_ACTIVE=docker dans docker-compose.yml.
# Ces valeurs remplacent celles de application.yml : on passe de H2 à PostgreSQL.
# "db" est le nom du service PostgreSQL dans docker-compose : Docker le résout comme un nom d'hôte.
spring:
  datasource:
    url: ${DB_URL:jdbc:postgresql://db:5432/secutrack}
    username: ${DB_USER:secutrack}
    password: ${DB_PASSWORD:secutrack}
```

Des propriétés typées

`backend/src/main/java/com/secutrack/config/SecutrackProperties.java`

```
/**
 * Propriétés propres à SecuTrack, lues dans application.yml :
 *
 * <pre>
 * secutrack:
 *   jwt:
 *     secret: ...            → jwt().secret()
 *     duree-minutes: 120     → jwt().dureeMinutes()   (Spring convertit kebab-case → camelCase)
 *   demo:
 *     actif: true            → demo().actif()
 *   cors-origines: [...]     → corsOrigines()
 * </pre>
 *
 * Avantage par rapport à plusieurs @Value("${...}") : tout est regroupé et typé dans un seul objet.
 */
// @ConfigurationProperties(prefix = "secutrack") : Spring remplit ce record avec toutes les
// propriétés qui commencent par "secutrack." (fichier YAML, variables d'environnement...)
@ConfigurationProperties(prefix = "secutrack")
public record SecutrackProperties(Jwt jwt, Demo demo, List<String> corsOrigines) {

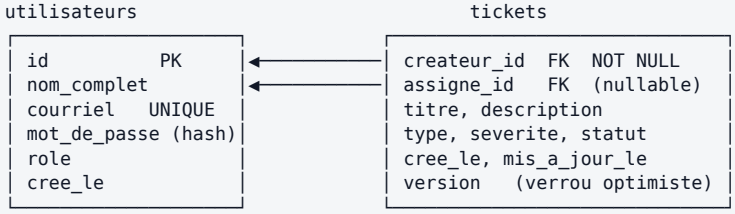
    /** Bloc secutrack.jwt : clé de signature et durée de validité des jetons. */
    public record Jwt(String secret, long dureeMinutes) {
    }

    /** Bloc secutrack.demo : création des comptes et tickets de démonstration au démarrage. */
    public record Demo(boolean actif, String motDePasse) {
    }
}
```

ddl-auto	Effet	Usage
`none`	Hibernate ne touche pas au schéma	Production
`validate`	Vérifie que les tables correspondent aux entités	Production avec Flyway
`update`	Crée ou modifie les tables sans supprimer de données	Développement (SecuTrack)
`create-drop`	Recrée tout au démarrage, supprime à l'arrêt	Tests

5. Modèle de données (JPA)

JPA (Jakarta Persistence) est la norme Java pour relier des objets à des tables : c'est un **ORM**. Hibernate en est l'implémentation : tu manipules des objets `Ticket`, Hibernate écrit le SQL.



Les annotations JPA utilisées

Annotation	Rôle	Exemple SecuTrack
`@Entity`	La classe correspond à une table	`Ticket`, `Utilisateur`
`@Table(name, indexes)`	Nom de table, index	`utilisateurs` (pas `user`, mot réservé)
`@Id` + `@GeneratedValue(IDENTITY)`	Clé primaire auto-incrémentée	`id`
`@Column(nullable, length, unique)`	Contraintes de colonne	`courriel` unique
`@Enumerated(EnumType.STRING)`	Enum stocké en texte	`statut`, `severite`, `role`
`@ManyToOne(fetch = LAZY)`	Clé étrangère vers une autre entité	`createur`, `assigne`
`@JoinColumn(name)`	Nom de la colonne de clé étrangère	`createur_id`
`@Version`	Verrouillage optimiste	`Ticket.version`
`@PrePersist` / `@PreUpdate`	Code exécuté avant INSERT / UPDATE	Dates de création et de mise à jour

L'entité Ticket

`backend/src/main/java/com/secutrack/ticket/Ticket.java`

```
/**
 * Entité centrale de l'application : un ticket de sécurité (table "tickets").
 *
 * Relations avec Utilisateur :
 * <pre>
 *  utilisateurs                tickets
 *  ┌────────────┐    1      N  ┌──────────────────────┐
 *  │ id (PK)    │◄─────────────│ createur_id (FK)     │  obligatoire
 *  │            │◄─────────────│ assigne_id  (FK)     │  optionnel
 *  └────────────┘              └──────────────────────┘
 * </pre>
 */
// @Entity : classe persistée en base (une instance = une ligne de la table)
@Entity
// @Table : nom de la table + index. Un index accélère les recherches sur une colonne
// (ici les filtres par statut et par sévérité du tableau de bord).
@Table(name = "tickets", indexes = {
        @Index(name = "idx_ticket_statut", columnList = "statut"),
        @Index(name = "idx_ticket_severite", columnList = "severite")
})
public class Ticket {
```

```
// @Id + @GeneratedValue(IDENTITY) : clé primaire auto-incrémentée par la base
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

// @Column(nullable = false, length = 150) → colonne VARCHAR(150) NOT NULL
// La même limite est vérifiée dans TicketRequest (@Size(max = 150)) pour renvoyer un 400 clair.
@Column(nullable = false, length = 150)
private String titre;

@Column(length = 4000)
private String description;

// @Enumerated(STRING) : stocke "INCIDENT" en texte, jamais la position 0, 1, 2 (EnumType.ORDINAL)
@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 20)
private TypeTicket type;

@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 20)
private Severite severite;

@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 20)
private Statut statut;

// @ManyToOne : plusieurs tickets → un utilisateur. C'est le côté qui porte la clé étrangère.
//   fetch = LAZY : l'utilisateur n'est chargé que si on y accède (par défaut, @ManyToOne est EAGER :
//                  il chargerait toujours l'utilisateur, même inutilement).
//   optional = false : un ticket a toujours un créateur.
// @JoinColumn : nom de la colonne de clé étrangère dans la table tickets.
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "createur_id", nullable = false)
private Utilisateur createur;

// Assigné optionnel : la colonne peut être NULL (ticket pas encore pris en charge)
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "assigne_id")
private Utilisateur assigne;

// Instant : un point précis dans le temps (UTC), sans ambiguïté de fuseau horaire
@Column(nullable = false, updatable = false)
private Instant creeLe;

private Instant misAJourLe;

// @Version : verrouillage optimiste. Hibernate incrémente ce compteur à chaque UPDATE et vérifie
// qu'il n'a pas changé entre la lecture et l'écriture. Si deux personnes modifient le même ticket
// en même temps, la seconde reçoit une erreur (409) au lieu d'écraser le travail de la première.
@Version
private Long version;

/** Constructeur vide obligatoire pour JPA (Hibernate l'utilise pour recréer l'objet depuis la base). */
protected Ticket() {
}

public Ticket(String titre, String description, TypeTicket type, Severite severite, Utilisateur createur) {
    this.titre = titre;
    this.description = description;
    this.type = type;
    this.severite = severite;
    this.createur = createur;
    this.statut = Statut.OUVERT;     // règle métier : tout ticket naît OUVERT
}

// @PrePersist : appelée automatiquement juste avant l'INSERT
@PrePersist
void avantCreation() {
    this.creeLe = Instant.now();
    this.misAJourLe = this.creeLe;
```

```
    }

    // @PreUpdate : appelée automatiquement juste avant chaque UPDATE
    @PreUpdate
    void avantMiseAJour() {
        this.misAJourLe = Instant.now();
    }

    // ---- Comportements métier ----
    // Mettre ces petites méthodes dans l'entité rend le service plus lisible : ticket.estFerme()

    public void modifier(String titre, String description, TypeTicket type, Severite severite) {
        this.titre = titre;
        this.description = description;
        this.type = type;
        this.severite = severite;
    }

    public boolean estFerme() {
        return statut == Statut.FERME;
    }

    /** Comparaison par courriel (unique) : fonctionne même avant que l'id soit généré (tests unitaires). */
    public boolean estCreePar(Utilisateur u) {
        return createur.getCourriel().equals(u.getCourriel());
    }

    public boolean estAssigneA(Utilisateur u) {
        return assigne != null && assigne.getCourriel().equals(u.getCourriel());
    }
```

Attention — N'utilise jamais `EnumType.ORDINAL` (la valeur par défaut si on omet `@Enumerated`) : la base stockerait 0, 1, 2… et ajouter une valeur au début de l'enum fausserait toutes les données existantes.

Le cycle de vie d'une entité et le dirty checking

Une entité chargée par `findById` dans une transaction est **managée** : Hibernate mémorise son état. À la fin de la méthode `@Transactional`, il compare l'état actuel à l'état de départ et génère l'UPDATE si quelque chose a changé. C'est pour ça que `TicketService.modifier()` ou `changerStatut()` n'appellent jamais `save()`.

Pourquoi des DTO plutôt que les entités ?

L'API ne renvoie jamais une entité. `TicketResponse` et `UtilisateurDto` sont des **records** qui choisissent les champs exposés. On évite ainsi d'exposer le hash du mot de passe, les boucles infinies de sérialisation entre entités liées, les `LazyInitializationException`, et on peut faire évoluer la base sans casser le contrat utilisé par Angular. La conversion est faite par `TicketMapper`, toujours dans le service.

6. Repositories et requêtes

Avec Spring Data JPA, un repository est une simple interface : Spring génère l'implémentation au démarrage. En étendant `JpaRepository<Ticket, Long>`, on hérite de `save`, `findById`, `findAll`, `existsById`, `deleteById`, de la pagination, etc.

backend/src/main/java/com/secutrack/ticket/TicketRepository.java

```
/**
 * Accès aux données des tickets. Spring Data JPA génère l'implémentation au démarrage.
 *
 * JpaRepository          : CRUD de base (save, findById, findAll, deleteById...) + pagination.
 * JpaSpecificationExecutor : recherche avec des critères construits dynamiquement (TicketSpecifications).
 */
public interface TicketRepository extends JpaRepository<Ticket, Long>, JpaSpecificationExecutor<Ticket> {

    /**
     * Recherche filtrée et paginée.
     *
     * @EntityGraph : charge createur et assigne DANS LA MÊME requête SQL (jointure).
     * Sans lui : 1 requête pour les 10 tickets de la page, puis 1 requête par ticket pour son
     * créateur, et encore 1 pour son assigné = jusqu'à 21 requêtes. C'est le fameux "problème N+1".
     */
    @Override
    @EntityGraph(attributePaths = {"createur", "assigne"})
    Page<Ticket> findAll(Specification<Ticket> spec, Pageable pageable);

    /** Même optimisation pour le détail d'un ticket. */
    @Override
    @EntityGraph(attributePaths = {"createur", "assigne"})
    Optional<Ticket> findById(Long id);

    /**
     * @Query : requête écrite à la main en JPQL. Le JPQL manipule des ENTITÉS et leurs champs
     * (Ticket t, t.statut), pas des tables et des colonnes.
     * "SELECT new ..." construit directement un record pour chaque ligne du résultat (projection).
     */
    @Query("SELECT new com.secutrack.ticket.dto.CompteParStatut(t.statut, COUNT(t)) "
         + "FROM Ticket t GROUP BY t.statut")
    List<CompteParStatut> compterParStatut();

    /** @Param("exclus") relie le paramètre Java au paramètre nommé :exclus de la requête. */
    @Query("SELECT new com.secutrack.ticket.dto.CompteParSeverite(t.severite, COUNT(t)) "
         + "FROM Ticket t WHERE t.statut NOT IN :exclus GROUP BY t.severite")
    List<CompteParSeverite> compterParSeverite(@Param("exclus") Collection<Statut> exclus);

    /**
     * Requête dérivée : Spring lit le nom de la méthode.
     * countBy + Assigne.Courriel + And + Statut + NotIn
     * → SELECT COUNT(*) FROM tickets t JOIN utilisateurs u ON t.assigne_id = u.id
     *   WHERE u.courriel = ? AND t.statut NOT IN (?, ?)
     */
    long countByAssigneCourrielAndStatutNotIn(String courriel, Collection<Statut> exclus);
}
```

Trois façons d'écrire une requête

Méthode	Exemple	Quand l'utiliser
Requête dérivée	`findByCourriel`, `countByAssigneCourrielAndStatutNotIn`	Critères simples et fixes
`@Query` en JPQL	`compterParStatut()` avec `SELECT new ...`	Agrégations, projections, jointures

Méthode	Exemple	Quand l'utiliser
`Specification`	`TicketSpecifications.avecFiltre()`	Critères optionnels combinés dynamiquement

La recherche dynamique

backend/src/main/java/com/secutrack/ticket/TicketSpecifications.java

```
/**
 * Construit dynamiquement le WHERE de la recherche : seuls les filtres renseignés sont appliqués.
 *
 * Une Specification est une condition réutilisable : (root, query, cb) -> condition
 *   - root  : l'entité interrogée (Ticket) ; root.get("statut") = la colonne statut
 *   - query : la requête en cours (rarement utilisée)
 *   - cb    : le CriteriaBuilder, qui fabrique les conditions (equal, like, or, lower...)
 *
 * C'est l'équivalent typé et sûr d'un "WHERE 1=1 AND ..." concaténé à la main
 * (et sans risque d'injection SQL : les valeurs sont toujours passées en paramètres).
 */
public final class TicketSpecifications {

    private TicketSpecifications() {
        // classe utilitaire : pas d'instance
    }

    public static Specification<Ticket> avecFiltre(FiltreTickets filtre) {
        List<Specification<Ticket>> conditions = new ArrayList<>();

        if (filtre.statut() != null) {
            // → WHERE statut = ?
            conditions.add((root, query, cb) -> cb.equal(root.get("statut"), filtre.statut()));
        }
        if (filtre.severite() != null) {
            conditions.add((root, query, cb) -> cb.equal(root.get("severite"), filtre.severite()));
        }
        if (filtre.type() != null) {
            conditions.add((root, query, cb) -> cb.equal(root.get("type"), filtre.type()));
        }
        if (filtre.assigneId() != null) {
            // root.get("assigne").get("id") : on suit la relation @ManyToOne → assigne_id = ?
            conditions.add((root, query, cb) -> cb.equal(root.get("assigne").get("id"), filtre.assigneId()));
        }
        if (StringUtils.hasText(filtre.recherche())) {
            // Recherche insensible à la casse dans le titre OU la description :
            // → WHERE LOWER(titre) LIKE '%vpn%' OR LOWER(description) LIKE '%vpn%'
            String motif = "%" + filtre.recherche().trim().toLowerCase() + "%";
            conditions.add((root, query, cb) -> cb.or(
                    cb.like(cb.lower(root.<String>get("titre")), motif),
                    cb.like(cb.lower(root.<String>get("description")), motif)));
        }
        // allOf : combine toutes les conditions avec AND (aucune condition = tous les tickets)
        return Specification.allOf(conditions);
    }
}
```

Attention — Le problème N+1 est le piège de performance numéro un en JPA. Sans `@EntityGraph`, afficher 10 tickets avec leur créateur et leur assigné provoquerait jusqu'à 21 requêtes SQL. Pour le voir de tes yeux, ajoute `spring.jpa.show-sql: true` dans `application.yml` et retire temporairement l'annotation.

7. Services et règles métier

Le service contient la logique métier : règles de gestion, vérifications, conversion entité/DTO. Il définit aussi les **transactions** : avec `@Transactional`, toute la méthode s'exécute dans une seule transaction, validée à la fin (commit) ou entièrement annulée si une `RuntimeException` est levée (rollback).

Les règles de SecuTrack

Action	USER	ADMIN	Où c'est vérifié
Consulter, créer un ticket	✓	✓	Être connecté (`SecuriteConfig`)
Modifier un ticket	son ticket	✓	`TicketService.modifier`
Passer OUVERT ↔ EN_COURS	créateur ou assigné	✓	`TicketService.changerStatut`
Passer à RESOLU ou FERME	✗	✓	`Statut.RESERVES_ADMIN`
Assigner	✗	✓	`@PreAuthorize` sur le contrôleur
Supprimer	✗	✓	Règle d'URL + `@PreAuthorize`
Modifier un ticket FERME	✗	✗	`verifierModifiable` → 409
Changer un rôle	✗	✓ sauf le sien	`UtilisateurService`

Contexte Hydro-Québec — Le frontend masque les boutons interdits, mais ce n'est qu'un confort d'affichage : n'importe qui peut appeler l'API avec curl ou Postman. Toutes les règles sont donc revérifiées dans le backend. En revue de code dans une équipe liée à la cybersécurité, c'est le premier réflexe attendu.

TicketService

`backend/src/main/java/com/secutrack/ticket/TicketService.java`

```
/**
 * Cœur métier des tickets. Toutes les règles de gestion sont ici (jamais dans le contrôleur) :
 * <ul>
 *   <li>tout utilisateur connecté peut consulter et créer des tickets ;</li>
 *   <li>le créateur ou un ADMIN peut modifier le contenu ;</li>
 *   <li>le créateur, l'assigné ou un ADMIN peut changer le statut,
 *       mais seul un ADMIN peut passer à RESOLU ou FERME ;</li>
 *   <li>un ticket FERME n'est plus modifiable ;</li>
 *   <li>seul un ADMIN peut assigner ou supprimer (aussi vérifié par @PreAuthorize dans le contrôleur).</li>
 * </ul>
 * Le frontend masque les boutons interdits, mais c'est ICI que la sécurité est réellement garantie :
 * un utilisateur peut toujours appeler l'API directement (curl, Postman).
 */
// @Service : bean de la couche métier
@Service
// @Transactional(readOnly = true) : par défaut, chaque méthode publique s'exécute dans une transaction
// en lecture seule. Les méthodes qui écrivent la surchargent avec @Transactional tout court.
@Transactional(readOnly = true)
public class TicketService {

    private final TicketRepository tickets;
    private final UtilisateurRepository utilisateurs;
    private final TicketMapper mapper;
```

```
// Injection par constructeur : Spring fournit les trois beans automatiquement
public TicketService(TicketRepository tickets, UtilisateurRepository utilisateurs, TicketMapper mapper) {
    this.tickets = tickets;
    this.utilisateurs = utilisateurs;
    this.mapper = mapper;
}

/** Pageable contient le numéro de page, la taille et le tri, construits par Spring depuis l'URL. */
public Page<TicketResponse> rechercher(FiltreTickets filtre, Pageable pageable) {
    // page.map() convertit chaque Ticket de la page en TicketResponse, en gardant les infos de pagination
    return tickets.findAll(TicketSpecifications.avecFiltre(filtre), pageable).map(mapper::toResponse);
}

public TicketResponse trouver(Long id) {
    return mapper.toResponse(charger(id));
}

// @Transactional : écriture. Si une exception non vérifiée survient, tout est annulé (rollback).
@Transactional
public TicketResponse creer(TicketRequest request, String courrielCourant) {
    Utilisateur createur = utilisateur(courrielCourant);   // le créateur vient du JWT, pas du client
    Ticket ticket = new Ticket(request.titre().trim(), request.description(), request.type(),
            request.severite(), createur);
    return mapper.toResponse(tickets.save(ticket));       // save() → INSERT
}

@Transactional
public TicketResponse modifier(Long id, TicketRequest request, String courrielCourant) {
    Ticket ticket = charger(id);
    Utilisateur courant = utilisateur(courrielCourant);
    verifierModifiable(ticket);
    if (!courant.estAdmin() && !ticket.estCreePar(courant)) {
        // AccessDeniedException → 403 (GlobalExceptionHandler)
        throw new AccessDeniedException("Seul le créateur du ticket ou un administrateur peut le modifier");
    }
    ticket.modifier(request.titre().trim(), request.description(), request.type(), request.severite());
    // Pas de save() : le ticket chargé est "managé" par Hibernate. À la fin de la transaction,
    // Hibernate compare son état avec celui du chargement et génère l'UPDATE (dirty checking).
    return mapper.toResponse(ticket);
}

@Transactional
public TicketResponse changerStatut(Long id, Statut nouveauStatut, String courrielCourant) {
    Ticket ticket = charger(id);
    Utilisateur courant = utilisateur(courrielCourant);
    verifierModifiable(ticket);
    if (Statut.RESERVES_ADMIN.contains(nouveauStatut) && !courant.estAdmin()) {
        throw new AccessDeniedException("Seul un administrateur peut passer un ticket à " + nouveauStatut);
    }
    if (!courant.estAdmin() && !ticket.estCreePar(courant) && !ticket.estAssigneA(courant)) {
        throw new AccessDeniedException("Vous n'êtes ni le créateur ni l'assigné de ce ticket");
    }
    ticket.setStatut(nouveauStatut);
    return mapper.toResponse(ticket);
}

/** assigneId null = retirer l'assignation. Le contrôle "ADMIN seulement" est fait par @PreAuthorize. */
@Transactional
public TicketResponse assigner(Long id, Long assigneId) {
    Ticket ticket = charger(id);
    verifierModifiable(ticket);
    Utilisateur assigne = (assigneId == null) ? null : utilisateurs.findById(assigneId)
            .orElseThrow(() -> new RessourceIntrouvableException("Utilisateur " + assigneId + " introuvable"));
    ticket.setAssigne(assigne);
    return mapper.toResponse(ticket);
}

@Transactional
public void supprimer(Long id) {
    if (!tickets.existsById(id)) {
```

```
            throw new RessourceIntrouvableException("Ticket " + id + " introuvable");   // → 404
        }
        tickets.deleteById(id);
    }
```

Les pièges de @Transactional

Piège	Explication
Méthode non publique	Spring entoure le service d'un proxy : l'annotation n'agit que sur les méthodes `public`.
Auto-invocation	Un appel `this.autreMethode()` ne passe pas par le proxy : l'annotation de `autreMethode` est ignorée.
Exception vérifiée	Seules les `RuntimeException` annulent la transaction. Nos exceptions métier en héritent.
`readOnly = true`	Désactive le dirty checking : un `setXxx()` dans une méthode en lecture seule n'est pas enregistré.

8. Contrôleurs et API REST

Le contrôleur est la porte d'entrée HTTP. Il reçoit la requête, valide l'entrée, appelle le service et construit la réponse. Il ne contient aucune règle métier.

Les endpoints

Verbe	URL	Accès	Réponse
POST	`/api/auth/register`	public	201 + jeton
POST	`/api/auth/login`	public	200 + jeton, 401 si invalide
GET	`/api/auth/moi`	connecté	profil
GET	`/api/tickets?statut=&severite=&type=&assigneId=&recherche=&page=&size=&sort=`	connecté	page de tickets
GET	`/api/tickets/statistiques`	connecté	compteurs
GET	`/api/tickets/{id}`	connecté	ticket ou 404
POST	`/api/tickets`	connecté	201 + Location
PUT	`/api/tickets/{id}`	créateur ou ADMIN	ticket
PATCH	`/api/tickets/{id}/statut`	selon les règles	ticket
PATCH	`/api/tickets/{id}/assignation`	ADMIN	ticket
DELETE	`/api/tickets/{id}`	ADMIN	204
GET	`/api/utilisateurs`	ADMIN	liste
PATCH	`/api/utilisateurs/{id}/role`	ADMIN	utilisateur

TicketController

backend/src/main/java/com/secutrack/ticket/TicketController.java

```
@Tag(name = "Tickets", description = "Gestion des tickets de sécurité")
// @RestController : les méthodes renvoient des objets convertis en JSON par Jackson
@RestController
// @RequestMapping : toutes les URL de cette classe commencent par /api/tickets
@RequestMapping("/api/tickets")
public class TicketController {

    private final TicketService service;

    public TicketController(TicketService service) {
        this.service = service;
    }

    @Operation(summary = "Rechercher les tickets (filtres, pagination et tri)",
               description = "Exemple : ?statut=OUVERT&severite=CRITIQUE&recherche=vpn&page=0&size=10&sort=creeLe,desc")
    // @GetMapping sans chemin : répond à GET /api/tickets
    @GetMapping
    public Page<TicketResponse> rechercher(
            // @ParameterObject (springdoc) : affiche chaque champ du filtre comme un paramètre dans Swagger
            @ParameterObject FiltreTickets filtre,
```

```
            // Pageable : Spring le construit depuis ?page=0&size=10&sort=creeLe,desc
            // @PageableDefault : valeurs utilisées si le client n'envoie rien
            @ParameterObject @PageableDefault(size = 10, sort = "creeLe", direction = Sort.Direction.DESC)
            Pageable pageable) {
        return service.rechercher(filtre, pageable);
    }

    @Operation(summary = "Statistiques pour le tableau de bord")
    // "/statistiques" est un chemin fixe : Spring le préfère à "/{id}", il n'y a donc pas de conflit
    @GetMapping("/statistiques")
    public StatistiquesResponse statistiques(@AuthenticationPrincipal Jwt jwt) {
        return service.statistiques(jwt.getSubject());
    }

    @Operation(summary = "Détail d'un ticket")
    // @PathVariable : lit {id} dans l'URL (/api/tickets/42 → id = 42)
    @GetMapping("/{id}")
    public TicketResponse trouver(@PathVariable Long id) {
        return service.trouver(id);
    }

    @Operation(summary = "Créer un ticket (statut initial : OUVERT)")
    @PostMapping
    public ResponseEntity<TicketResponse> creer(
            // @Valid : vérifie les contraintes de TicketRequest (@NotBlank, @Size...) ; échec → 400
            // @RequestBody : convertit le JSON reçu en TicketRequest
            @Valid @RequestBody TicketRequest request,
            // @AuthenticationPrincipal : le JWT de l'utilisateur connecté ; getSubject() = son courriel
            @AuthenticationPrincipal Jwt jwt) {
        TicketResponse cree = service.creer(request, jwt.getSubject());
        // Bonne pratique REST : 201 Created + header "Location: /api/tickets/{id}" de la nouvelle ressource
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").buildAndExpand(cree.id()).toUri();
        // ResponseEntity : contrôle total sur le statut HTTP, les headers et le corps de la réponse
        return ResponseEntity.created(location).body(cree);
    }

    @Operation(summary = "Modifier un ticket (créateur ou ADMIN)")
    // @PutMapping : remplacement complet des champs modifiables
    @PutMapping("/{id}")
    public TicketResponse modifier(@PathVariable Long id, @Valid @RequestBody TicketRequest request,
                                   @AuthenticationPrincipal Jwt jwt) {
        return service.modifier(id, request, jwt.getSubject());
    }

    @Operation(summary = "Changer le statut (RESOLU et FERME : ADMIN seulement)")
    // @PatchMapping : modification partielle (un seul champ)
    @PatchMapping("/{id}/statut")
    public TicketResponse changerStatut(@PathVariable Long id, @Valid @RequestBody StatutRequest request,
                                        @AuthenticationPrincipal Jwt jwt) {
        return service.changerStatut(id, request.statut(), jwt.getSubject());
    }

    @Operation(summary = "Assigner ou désassigner un ticket (ADMIN)")
    @PatchMapping("/{id}/assignation")
    // @PreAuthorize : vérifié AVANT d'exécuter la méthode ; un USER reçoit 403
    // (actif grâce à @EnableMethodSecurity dans SecuriteConfig)
    @PreAuthorize("hasRole('ADMIN')")
    public TicketResponse assigner(@PathVariable Long id, @Valid @RequestBody AssignationRequest request) {
        return service.assigner(id, request.assigneId());
    }

    @Operation(summary = "Supprimer un ticket (ADMIN)")
    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    // @ResponseStatus(NO_CONTENT) : 204 = succès sans corps de réponse
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void supprimer(@PathVariable Long id) {
        service.supprimer(id);
    }
}
```

Exemple d'échange complet

```
POST /api/tickets
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{ "titre": "Accès VPN pour un consultant", "type": "DEMANDE_ACCES", "severite": "FAIBLE" }

HTTP/1.1 201 Created
Location: http://localhost:8080/api/tickets/6

{ "id": 6, "titre": "Accès VPN pour un consultant", "description": null,
  "type": "DEMANDE_ACCES", "severite": "FAIBLE", "statut": "OUVERT",
  "createur": { "id": 2, "nomComplet": "Bruno Analyste", "courriel": "analyste@secutrack.local" },
  "assigne": null, "creeLe": "2026-10-19T13:05:12Z", "misAJourLe": "2026-10-19T13:05:12Z" }
```

La réponse paginée de `GET /api/tickets` a la forme `{ "content": [ ...tickets ], "page": { "size": 10, "number": 0, "totalElements": 5, "totalPages": 1 } }`, grâce à la propriété `spring.data.web.pageable.serialization-mode: via-dto`.

9. Validation et gestion des erreurs

La validation des entrées

Les règles sont déclarées sur les records de requête avec Bean Validation. Elles ne sont vérifiées que si le contrôleur place `@Valid` devant `@RequestBody`.

backend/src/main/java/com/secutrack/ticket/dto/TicketRequest.java

```
/**
 * Corps de POST /api/tickets (création) et PUT /api/tickets/{id} (modification).
 * Le client n'envoie ni le statut, ni le créateur : le serveur les décide (sécurité).
 */
public record TicketRequest(
        // @NotBlank : obligatoire et pas seulement des espaces
        @NotBlank(message = "Le titre est obligatoire")
        // @Size(max = 150) : aligné sur @Column(length = 150) de l'entité Ticket
        @Size(max = 150, message = "150 caractères maximum")
        String titre,

        // Pas de @NotBlank : la description est optionnelle
        @Size(max = 4000, message = "4000 caractères maximum")
        String description,

        // @NotNull : pour un enum (ou un objet), "obligatoire" se dit @NotNull, pas @NotBlank
        @NotNull(message = "Le type est obligatoire")
        TypeTicket type,

        @NotNull(message = "La sévérité est obligatoire")
        Severite severite) {
}
```

Annotation	Vérifie que…
`@NotNull`	la valeur n'est pas null (enums, objets, nombres)
`@NotBlank`	la chaîne contient au moins un caractère non-espace
`@Size(min, max)`	la longueur est dans l'intervalle
`@Email`	le format de courriel est valide
`@Pattern(regexp)`	la chaîne respecte une expression régulière
`@Positive`	le nombre est strictement positif (null accepté)

La gestion centralisée des erreurs

backend/src/main/java/com/secutrack/common/GlobalExceptionHandler.java

```
/**
 * Gestion centralisée des erreurs : au lieu de mettre des try/catch dans chaque contrôleur,
 * les exceptions "remontent" jusqu'ici et sont converties en réponse HTTP propre.
 *
 * Toutes les erreurs sont renvoyées au format standard ProblemDetail (RFC 9457) :
 * <pre>
 * { "type": "about:blank", "title": "Requête invalide", "status": 400,
 *   "detail": "...", "instance": "/api/tickets", "erreurs": { "titre": "Le titre est obligatoire" } }
 * </pre>
 * Angular reçoit donc toujours la même structure, quelle que soit l'erreur.
 */
// @RestControllerAdvice : cette classe intercepte les exceptions levées par TOUS les contrôleurs
@RestControllerAdvice
public class GlobalExceptionHandler {
```

```
// Logger SLF4J : écrit dans les journaux du serveur (console, fichiers), jamais vers le client
private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

// @ExceptionHandler(X.class) : cette méthode est appelée quand une exception de type X est levée.
// Spring choisit toujours le handler le plus précis (sous-classe avant classe parente).
@ExceptionHandler(RessourceIntrouvableException.class)
public ProblemDetail introuvable(RessourceIntrouvableException e) {
    return probleme(HttpStatus.NOT_FOUND, "Ressource introuvable", e.getMessage());          // 404
}

@ExceptionHandler(BusinessException.class)
public ProblemDetail regleMetier(BusinessException e) {
    return probleme(HttpStatus.CONFLICT, "Règle métier non respectée", e.getMessage());      // 409
}

/** Levée par Hibernate quand @Version détecte que deux personnes ont modifié le même ticket. */
@ExceptionHandler(ObjectOptimisticLockingFailureException.class)
public ProblemDetail modificationConcurrente(ObjectOptimisticLockingFailureException e) {
    return probleme(HttpStatus.CONFLICT, "Modification concurrente",
            "Le ticket a été modifié par quelqu'un d'autre. Rechargez-le puis recommencez.");
}

/**
 * Erreurs de validation (400), avec le détail champ par champ.
 * BindException couvre @Valid sur un @RequestBody (sa sous-classe MethodArgumentNotValidException)
 * ET les filtres reçus en paramètres d'URL (?statut=INCONNU).
 */
@ExceptionHandler(BindException.class)
public ProblemDetail requeteInvalide(BindException e) {
    ProblemDetail p = probleme(HttpStatus.BAD_REQUEST, "Requête invalide",
            "Certains champs ne respectent pas les règles de validation");
    Map<String, String> erreurs = new LinkedHashMap<>();
    e.getBindingResult().getFieldErrors()
            .forEach(err -> erreurs.putIfAbsent(err.getField(), err.getDefaultMessage()));
    p.setProperty("erreurs", erreurs);   // propriété personnalisée ajoutée au JSON
    return p;
}

/** Paramètre d'URL invalide : id non numérique, tri sur une colonne inconnue (?sort=xyz)... → 400 */
@ExceptionHandler({HandlerMethodValidationException.class, MethodArgumentTypeMismatchException.class,
        PropertyReferenceException.class})
public ProblemDetail parametreInvalide(Exception e) {
    return probleme(HttpStatus.BAD_REQUEST, "Paramètre invalide", e.getMessage());
}

/** Corps JSON illisible : virgule manquante, ou valeur d'énumération inconnue ("severite": "XYZ") → 400 */
@ExceptionHandler(HttpMessageNotReadableException.class)
public ProblemDetail corpsIllisible(HttpMessageNotReadableException e) {
    return probleme(HttpStatus.BAD_REQUEST, "Corps de requête illisible",
            "JSON mal formé ou valeur d'énumération inconnue");
}

/** Mauvais courriel ou mot de passe au login → 401. Message volontairement vague (sécurité). */
@ExceptionHandler(AuthenticationException.class)
public ProblemDetail authentification(AuthenticationException e) {
    return probleme(HttpStatus.UNAUTHORIZED, "Authentification refusée", "Courriel ou mot de passe invalide");
}

/** Levée par @PreAuthorize ou par nos règles dans TicketService → 403. */
@ExceptionHandler(AccessDeniedException.class)
public ProblemDetail accesRefuse(AccessDeniedException e) {
    return probleme(HttpStatus.FORBIDDEN, "Accès refusé", e.getMessage());
}

/**
 * Filet de sécurité pour tout le reste (bug imprévu) → 500.
 * On journalise la trace complète côté serveur, mais on ne l'expose JAMAIS au client :
 * une trace Java révèle la structure interne de l'application (information utile à un attaquant).
 */
@ExceptionHandler(Exception.class)
```

```
    public ProblemDetail inattendue(Exception e) {
        log.error("Erreur non gérée", e);
        return probleme(HttpStatus.INTERNAL_SERVER_ERROR, "Erreur interne", "Une erreur inattendue est survenue");
    }

    /** Construit un ProblemDetail avec un statut, un titre court et un détail lisible. */
    private ProblemDetail probleme(HttpStatus statut, String titre, String detail) {
        ProblemDetail p = ProblemDetail.forStatusAndDetail(statut, detail);
        p.setTitle(titre);
        return p;
    }
}
```

Les codes HTTP de SecuTrack

Code	Quand	Source
200 / 201 / 204	Succès, création, suppression	Contrôleurs
400	Validation, JSON illisible, paramètre invalide	`BindException`, `HttpMessageNotReadableException`…
401	Pas de jeton valide, ou mauvais mot de passe au login	Filtre JWT, `AuthenticationException`
403	Rôle insuffisant ou règle de droit non respectée	Règles d'URL, `@PreAuthorize`, `AccessDeniedException`
404	Ticket ou utilisateur introuvable	`RessourceIntrouvableException`
409	Ticket fermé, courriel déjà utilisé, modification concurrente	`BusinessException`, `@Version`
500	Bug imprévu (trace dans les logs seulement)	Handler `Exception.class`

10. Sécurité : Spring Security et JWT

Les deux questions de la sécurité

L'authentification répond à « qui es-tu ? » : SecuTrack la fait avec un JWT. **L'autorisation** répond à « as-tu le droit ? » : elle repose sur les rôles USER et ADMIN. Un échec d'authentification donne **401**, un échec d'autorisation donne **403**.

Qu'est-ce qu'un JWT ?

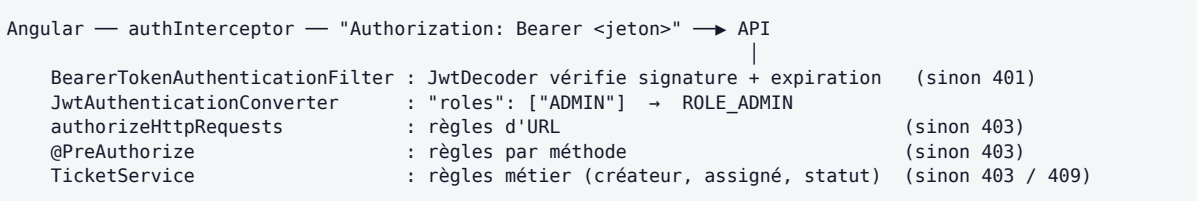
Un JSON Web Token est une chaîne en trois parties séparées par des points : `en-tête.contenu.signature`. Le contenu (les *claims*) est simplement encodé en Base64 : **n'importe qui peut le lire** (sur jwt.io par exemple), donc on n'y met jamais de secret. La signature, calculée avec la clé secrète du serveur, garantit qu'il n'a pas été modifié : si un utilisateur remplace `"roles": ["USER"]` par `["ADMIN"]`, la signature ne correspond plus et le jeton est refusé.

```
{                                       ← contenu d'un jeton SecuTrack
  "iss": "secutrack",                   émetteur
  "sub": "analyste@secutrack.local",    sujet : qui est l'utilisateur
  "roles": ["USER"],                    rôles → autorité ROLE_USER
  "nom": "Bruno Analyste",
  "iat": 1760878800,                    émis le
  "exp": 1760886000                     expire le (120 minutes plus tard)
}
```

Le flux de connexion



Le flux d'une requête protégée



La configuration

`backend/src/main/java/com/secutrack/config/SecuriteConfig.java`

```
/**
 * Configuration centrale de la sécurité.
 *
 * Spring Security fonctionne comme une chaîne de filtres : chaque requête HTTP traverse
 * ces filtres AVANT d'arriver au contrôleur. Ils répondent à deux questions :
 *   1. Authentification : qui es-tu ?       → vérification du JWT  (échec = 401 Unauthorized)
 *   2. Autorisation     : as-tu le droit ?  → vérification du rôle (échec = 403 Forbidden)
 */
```

```
@Configuration
// @EnableWebSecurity : active notre configuration de sécurité web personnalisée
@EnableWebSecurity
// @EnableMethodSecurity : active @PreAuthorize("hasRole('ADMIN')") sur les méthodes des contrôleurs
@EnableMethodSecurity
public class SecuriteConfig {

    /** URL accessibles sans être connecté. */
    private static final String[] ROUTES_PUBLIQUES = {
            "/api/auth/login", "/api/auth/register",                        // impossible d'exiger un jeton pour e
                n obtenir un
            "/actuator/health/**",                                          // sonde de santé (Docker, Kubernetes)
            "/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html"         // documentation de l'API
    };

    /**
     * La chaîne de filtres de sécurité. Depuis Spring Security 6, on la configure
     * en déclarant un bean SecurityFilterChain (l'ancienne classe WebSecurityConfigurerAdapter n'existe plus).
     */
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            // CSRF protège les applications qui utilisent des cookies de session.
            // Notre API est "stateless" (le jeton est envoyé dans un header) : on peut le désactiver.
            .csrf(csrf -> csrf.disable())
            // Autorise Angular (localhost:4200) à appeler l'API : voir corsConfigurationSource() plus bas
            .cors(Customizer.withDefaults())
            // STATELESS : Spring ne crée jamais de session HTTP ; chaque requête doit porter son JWT
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            // Règles d'autorisation par URL, lues dans l'ordre : la première qui correspond gagne
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(ROUTES_PUBLIQUES).permitAll()
                .requestMatchers(HttpMethod.DELETE, "/api/tickets/**").hasRole("ADMIN")
                .requestMatchers("/api/utilisateurs/**").hasRole("ADMIN")
                .anyRequest().authenticated())                              // tout le reste : connecté
            // L'API joue le rôle de "resource server" : elle lit le header "Authorization: Bearer <jeton>",
            // vérifie le jeton avec le JwtDecoder (JwtConfig) puis applique le convertisseur de rôles
            .oauth2ResourceServer(oauth -> oauth
                .jwt(jwt -> jwt.jwtAuthenticationConverter(jwtAuthenticationConverter())))
            // Réponses en cas d'échec au niveau des filtres
            .exceptionHandling(e -> e
                .authenticationEntryPoint(new BearerTokenAuthenticationEntryPoint())   // pas de jeton valide → 401
                .accessDeniedHandler(new BearerTokenAccessDeniedHandler()));           // mauvais rôle → 403
        return http.build();
    }

    /**
     * Notre JWT contient "roles": ["ADMIN"]. Spring, lui, raisonne en "autorités" préfixées : "ROLE_ADMIN".
     * Ce convertisseur fait le lien, ce qui permet d'écrire hasRole('ADMIN') dans les règles.
     */
    @Bean
    public JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtGrantedAuthoritiesConverter roles = new JwtGrantedAuthoritiesConverter();
        roles.setAuthoritiesClaimName("roles");     // nom de la claim à lire dans le jeton
        roles.setAuthorityPrefix("ROLE_");          // préfixe attendu par hasRole()
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(roles);
        return converter;
    }

    /**
     * Utilisé au login (AuthService) pour vérifier le couple courriel / mot de passe.
     * Spring le construit automatiquement à partir de notre UserDetailsService
     * (UtilisateurDetailsService) et du PasswordEncoder ci-dessous.
     */
    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration configuration) throws Exception {
        return configuration.getAuthenticationManager();
    }

    /**
```

```
     * BCrypt : algorithme de hachage conçu pour les mots de passe (lent volontairement, avec un "sel"
     * aléatoire). On ne stocke jamais un mot de passe en clair, seulement son hash.
     */
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    /**
     * CORS : par défaut, le navigateur bloque un appel de http://localhost:4200 (Angular)
     * vers http://localhost:8080 (API), car ce sont deux "origines" différentes.
     * On autorise explicitement les origines listées dans secutrack.cors-origines.
     * (Avec docker compose, nginx sert Angular et l'API sur la même origine : CORS n'intervient pas.)
     */
    @Bean
    public CorsConfigurationSource corsConfigurationSource(SecutrackProperties proprietes) {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(proprietes.corsOrigines());
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
        config.setAllowedHeaders(List.of("Authorization", "Content-Type"));
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/api/**", config);
        return source;
    }
}
```

Émettre et vérifier les jetons

backend/src/main/java/com/secutrack/config/JwtConfig.java

```
/**
 * Configuration des JWT (JSON Web Token).
 *
 * Un JWT a trois parties séparées par des points : en-tête.contenu.signature
 *   - en-tête   : l'algorithme de signature (ici HS256)
 *   - contenu   : les "claims" (courriel, rôles, date d'expiration...)
 *   - signature : calculée avec la clé secrète. Si quelqu'un modifie le contenu
 *                 (par exemple pour se donner le rôle ADMIN), la signature ne correspond plus
 *                 et le jeton est refusé.
 *
 * HS256 est un algorithme symétrique : la même clé secrète sert à signer ET à vérifier.
 * La bibliothèque Nimbus est fournie par Spring Security (aucune dépendance en plus).
 */
// @Configuration : classe qui déclare des beans avec des méthodes @Bean
@Configuration
public class JwtConfig {

    // @Bean : l'objet retourné devient un bean géré par Spring, injectable partout.
    // On utilise @Bean (et pas @Component) car SecretKey, JwtEncoder et JwtDecoder
    // sont des classes de bibliothèques qu'on ne peut pas annoter nous-mêmes.
    @Bean
    public SecretKey cleJwt(SecutrackProperties proprietes) {
        return creerCle(proprietes.jwt().secret());
    }

    /** Signe les jetons au login (utilisé par JwtService). */
    @Bean
    public JwtEncoder jwtEncoder(SecretKey cleJwt) {
        return new NimbusJwtEncoder(new ImmutableSecret<>(cleJwt));
    }

    /**
     * Vérifie les jetons à chaque requête : signature valide + date d'expiration non dépassée.
     * Spring Security l'utilise automatiquement grâce à .oauth2ResourceServer() dans SecuriteConfig.
     */
    @Bean
    public JwtDecoder jwtDecoder(SecretKey cleJwt) {
        return NimbusJwtDecoder.withSecretKey(cleJwt)
                .macAlgorithm(MacAlgorithm.HS256)
                .build();
```

```
    }

    /**
     * Construit la clé à partir du secret de la configuration.
     * Méthode statique pour pouvoir l'utiliser dans un test unitaire, sans démarrer Spring.
     */
    public static SecretKey creerCle(String secret) {
        byte[] octets = secret.getBytes(StandardCharsets.UTF_8);
        // HS256 exige une clé d'au moins 256 bits = 32 octets. On bloque le démarrage sinon.
        if (octets.length < 32) {
            throw new IllegalStateException("secutrack.jwt.secret doit contenir au moins 32 caractères");
        }
        return new SecretKeySpec(octets, "HmacSHA256");
    }
}
```

Les choix de sécurité du projet

Choix	Pourquoi
Mots de passe hachés avec BCrypt	Une fuite de la base ne révèle pas les mots de passe
Inscription toujours en USER	Personne ne peut s'inscrire administrateur
Message de login vague	« Courriel ou mot de passe invalide » ne révèle pas quels comptes existent
Secret JWT de 32 caractères minimum, hors de Git	Clé HS256 robuste, fournie par variable d'environnement
Créateur pris dans le JWT, jamais dans le JSON	Le client ne peut pas créer un ticket au nom d'un autre
Traces d'erreur jamais renvoyées au client	Pas d'information interne exploitable par un attaquant
Actuator limité à health et info	`/actuator/env` ou `/heapdump` exposeraient des secrets
Conteneur backend sans root, en-têtes nginx	Réduction de la surface d'attaque

Contexte Hydro-Québec — En entreprise, l'authentification passe souvent par un fournisseur d'identité central (SSO, OIDC). L'API SecuTrack est déjà un « resource server » : pour utiliser un tel fournisseur, il suffirait de remplacer le `JwtDecoder` de `JwtConfig` par la propriété `spring.security.oauth2.resourceserver.jwt.issuer-uri`. Le reste du code, y compris `@PreAuthorize`, ne changerait pas.

Attention — Côté Angular, le jeton est stocké dans le `localStorage` pour survivre à un rafraîchissement de page. C'est simple, mais un script injecté (faille XSS) pourrait le lire. Les alternatives sont un cookie HttpOnly ou un fournisseur d'identité ; c'est une bonne discussion à avoir avec une équipe cybersécurité.

11. Tests du backend

Fichier	Type	Ce qu'il vérifie
`TicketServiceTest`	Unitaire (Mockito)	Règles métier : statut initial, droits USER/ADMIN, ticket fermé
`JwtServiceTest`	Unitaire (sans Spring)	Contenu du jeton, refus d'un jeton signé avec une autre clé
`TicketApiIntegrationTest`	Intégration (`@SpringBootTest`)	401, 403, 400, inscription, création, suppression avec de vrais JWT
`SecutrackApplicationTests`	Fumée	L'application démarre

Annotation	Rôle
`@Test`, `@DisplayName`, `@BeforeEach`	Test JUnit 5, nom lisible, préparation avant chaque test
`@ExtendWith(MockitoExtension.class)`	Active Mockito sans démarrer Spring
`@Mock` / `@Spy` / `@InjectMocks`	Faux objet / vrai objet espionné / classe testée avec ses dépendances
`@SpringBootTest`	Démarre toute l'application
`@AutoConfigureMockMvc`	Fournit `MockMvc` pour simuler des requêtes HTTP
`@Transactional` (sur un test)	Rollback automatique après chaque test

Un test unitaire

```
@Test
@DisplayName("un USER ne peut pas passer un ticket à RESOLU")
void userNePeutPasResoudre() {
    when(tickets.findById(1L)).thenReturn(Optional.of(ticketDeBruno()));
    when(utilisateurs.findByCourriel("bruno@test")).thenReturn(Optional.of(bruno));

    // assertThatThrownBy : vérifie que l'appel lève bien l'exception attendue
    assertThatThrownBy(() -> service.changerStatut(1L, Statut.RESOLU, "bruno@test"))
            .isInstanceOf(AccessDeniedException.class);
}

@Test
@DisplayName("un ADMIN peut passer un ticket à RESOLU")
void adminPeutResoudre() {
    when(tickets.findById(1L)).thenReturn(Optional.of(ticketDeBruno()));
    when(utilisateurs.findByCourriel("admin@test")).thenReturn(Optional.of(admin));

    TicketResponse r = service.changerStatut(1L, Statut.RESOLU, "admin@test");

    assertThat(r.statut()).isEqualTo(Statut.RESOLU);
}
```

Un test d'intégration de bout en bout

```
@Test
void seulLAdminPeutSupprimer() throws Exception {
    String jetonUser = inscrire("bruno@test.local");
    String jetonAdmin = connecter("admin@test.local");
    long id = creerTicket(jetonUser);

    mockMvc.perform(delete("/api/tickets/" + id).header("Authorization", "Bearer " + jetonUser))
            .andExpect(status().isForbidden());
```

```
        mockMvc.perform(delete("/api/tickets/" + id).header("Authorization", "Bearer " + jetonAdmin))
                .andExpect(status().isNoContent());
    }
```

Ce test passe par toute la chaîne réelle : inscription, login, génération et vérification des JWT, filtres de sécurité, validation, service et base H2. C'est le type de test qui protège le mieux une application sécurisée.

12. Frontend Angular : les fondations

Les concepts d'Angular moderne utilisés

Concept	Rôle	Exemple SecuTrack
Composant standalone	Classe + template + style, déclare ses propres `imports`	Toutes les pages
`inject()`	Injection de dépendance (équivalent du constructeur Spring)	`inject(TicketService)`
`signal()`	Valeur réactive ; l'affichage se met à jour quand elle change	`lignes`, `statut`, `session`
`computed()`	Valeur dérivée d'autres signals	`estAdmin`, `colonnes`, `peutModifier`
`effect()`	Code relancé quand un signal lu change	Rechargement du tableau de bord
`input()`	Donnée reçue de l'extérieur (ici, de l'URL)	`id` du ticket, `retour`
Reactive Forms	Formulaire déclaré en TypeScript avec ses validateurs	Connexion, ticket
Observable / `subscribe()`	Flux asynchrone ; la requête part au `subscribe()`	Tous les appels HTTP
Intercepteur	Modifie chaque requête HTTP	Ajout du JWT
Guard	Autorise ou redirige avant d'afficher une page	`authGuard`, `adminGuard`
Lazy loading	Une page n'est téléchargée qu'à la première visite	`loadComponent` dans les routes
`@if` / `@for`	Blocs de contrôle des templates	Affichage selon le rôle

La configuration et les routes

frontend/src/app/app.config.ts

```
/**
 * Configuration globale de l'application (l'équivalent des @Bean de Spring côté Angular).
 * Chaque "provider" rend un service disponible pour toute l'application.
 */
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    // Routage. withComponentInputBinding() : les paramètres d'URL (:id, ?retour=)
    // arrivent directement dans les input() des composants
    provideRouter(routes, withComponentInputBinding()),
    // HttpClient pour appeler l'API. withInterceptors : chaque requête passe par authInterceptor,
    // qui ajoute le JWT (équivalent d'un filtre côté Spring)
    provideHttpClient(withFetch(), withInterceptors([authInterceptor])),
    // <mat-icon> utilise la police Material Symbols (hébergée localement, voir angular.json)
    { provide: MAT_ICON_DEFAULT_OPTIONS, useValue: { fontSet: 'material-symbols-outlined' } },
  ],
};
```

frontend/src/app/app.routes.ts

```
/**
 * Table de routage : quelle URL affiche quel composant.
```

```
 *
 * - loadComponent : la page est chargée à la demande (lazy loading), le bundle initial reste léger.
 * - canActivate   : "guards" exécutés avant d'afficher la page (voir core/auth/auth.guards.ts).
 * - children      : toutes les routes enfants héritent du guard du parent (authGuard).
 * - title         : texte de l'onglet du navigateur.
 */
export const routes: Routes = [
  {
    path: 'connexion',
    canActivate: [invitesGuard],
    title: 'Connexion — SecuTrack',
    loadComponent: () => import('./pages/connexion/connexion').then((m) => m.Connexion),
  },
  {
    path: 'inscription',
    canActivate: [invitesGuard],
    title: 'Inscription — SecuTrack',
    loadComponent: () => import('./pages/inscription/inscription').then((m) => m.Inscription),
  },
  {
    path: '',
    canActivate: [authGuard],
    children: [
      {
        path: '',
        title: 'Tableau de bord — SecuTrack',
        loadComponent: () =>
          import('./pages/tableau-de-bord/tableau-de-bord').then((m) => m.TableauDeBord),
      },
      {
        path: 'tickets/nouveau',
        title: 'Nouveau ticket — SecuTrack',
        loadComponent: () =>
          import('./pages/ticket-formulaire/ticket-formulaire').then((m) => m.TicketFormulaire),
      },
      {
        path: 'tickets/:id',
        title: 'Ticket — SecuTrack',
        loadComponent: () =>
          import('./pages/ticket-detail/ticket-detail').then((m) => m.TicketDetail),
      },
      {
        path: 'tickets/:id/modifier',
        title: 'Modifier le ticket — SecuTrack',
        loadComponent: () =>
          import('./pages/ticket-formulaire/ticket-formulaire').then((m) => m.TicketFormulaire),
      },
      {
        path: 'utilisateurs',
        canActivate: [adminGuard],
        title: 'Utilisateurs — SecuTrack',
        loadComponent: () =>
          import('./pages/utilisateurs/utilisateurs').then((m) => m.Utilisateurs),
      },
    ],
  },
  // '**' : toute URL inconnue renvoie au tableau de bord
  { path: '**', redirectTo: '' },
];
```

La session : AuthService

`frontend/src/app/core/auth/auth.service.ts`

```
/**
 * Gère la session : jeton JWT + utilisateur courant, exposés sous forme de signals.
 *
 * Note de sécurité : le jeton est conservé dans le localStorage pour rester connecté après
 * un rafraîchissement. C'est simple, mais un script injecté (XSS) pourrait le lire. En entreprise,
 * on préfère souvent un cookie HttpOnly ou un fournisseur d'identité (OIDC).
```

```
 */
// @Injectable({ providedIn: 'root' }) : service singleton, une seule instance pour toute l'application
// (l'équivalent d'un @Service Spring)
@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly http = inject(HttpClient);
  private readonly router = inject(Router);

  // signal() : valeur réactive. Quand elle change (set), tout ce qui la lit est mis à jour
  // automatiquement : templates, computed(), effect(). C'est le cœur de la réactivité d'Angular moderne.
  private readonly session = signal<AuthResponse | null>(this.lireSession());

  // computed() : valeur DÉRIVÉE d'autres signals, recalculée automatiquement.
  // Ex. : quand session change, estAdmin se met à jour, et la barre d'outils affiche ou masque "Utilisateurs".

  readonly utilisateur = computed(() => this.session()?.utilisateur ?? null);
  readonly estConnecte = computed(() => this.session() !== null);
  readonly estAdmin = computed(() => this.utilisateur()?.role === 'ADMIN');

  connecter(request: LoginRequest): Observable<AuthResponse> {
    return (
      this.http
        .post<AuthResponse>('/api/auth/login', request)
        // pipe(tap(...)) : exécute un effet de bord (enregistrer la session) sans modifier la réponse
        .pipe(tap((reponse) => this.ouvrirSession(reponse)))
    );
  }

  inscrire(request: RegisterRequest): Observable<AuthResponse> {
    return this.http
      .post<AuthResponse>('/api/auth/register', request)
      .pipe(tap((reponse) => this.ouvrirSession(reponse)));
  }

  /** Jeton valide, ou null s'il est absent ou expiré (la session est alors fermée). */
  jeton(): string | null {
    const session = this.session();
    if (!session) {
      return null;
    }
    if (new Date(session.expiresAt).getTime() <= Date.now()) {
      this.fermerSession();
      return null;
    }
    return session.token;
  }

  deconnecter(): void {
    this.fermerSession();
    void this.router.navigate(['/connexion']);
  }

  private ouvrirSession(reponse: AuthResponse): void {
    localStorage.setItem(CLE_SESSION, JSON.stringify(reponse));
    this.session.set(reponse);
  }

  private fermerSession(): void {
    localStorage.removeItem(CLE_SESSION);
    this.session.set(null);
  }

  private lireSession(): AuthResponse | null {
    try {
      const brut = localStorage.getItem(CLE_SESSION);
      if (!brut) {
        return null;
      }
      const session = JSON.parse(brut) as AuthResponse;
      return new Date(session.expiresAt).getTime() > Date.now() ? session : null;
    } catch {
```

```
      return null;
    }
  }
}
```

L'intercepteur JWT

frontend/src/app/core/auth/auth.interceptor.ts

```
/**
 * Ajoute "Authorization: Bearer <jeton>" à chaque appel vers /api,
 * et ferme la session si l'API répond 401 (jeton expiré ou invalide).
 */
// HttpInterceptorFn : fonction appelée pour CHAQUE requête HttpClient.
// req = la requête sortante ; next(req) = l'envoie et renvoie un Observable de la réponse.
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const jeton = auth.jeton();

  // Les requêtes HttpClient sont immuables : clone() crée une copie avec le header en plus
  const requete =
    jeton && req.url.startsWith('/api/')
      ? req.clone({ setHeaders: { Authorization: `Bearer ${jeton}` } })
      : req;

  return next(requete).pipe(
    // catchError : intercepte les erreurs HTTP de la réponse
    catchError((err: unknown) => {
      const estAppelAuth =
        req.url.startsWith('/api/auth/login') || req.url.startsWith('/api/auth/register');
      if (err instanceof HttpErrorResponse && err.status === 401 && !estAppelAuth) {
        auth.deconnecter();
      }
      // On relance l'erreur pour que le composant appelant puisse aussi l'afficher
      return throwError(() => err);
    }),
  );
};
```

Les guards

frontend/src/app/core/auth/auth.guards.ts

```
/*
 * Guards : fonctions exécutées par le routeur AVANT d'afficher une page.
 * Elles renvoient true (accès autorisé) ou un UrlTree (redirection vers une autre page).
 *
 * Attention : un guard améliore l'expérience utilisateur, il ne SÉCURISE rien.
 * Le code Angular s'exécute dans le navigateur et peut être contourné ; la vraie sécurité
 * est dans le backend (Spring Security, @PreAuthorize, TicketService).
 */

/** Routes réservées aux utilisateurs connectés ; mémorise l'URL demandée pour y revenir. */
export const authGuard: CanActivateFn = (_route, state) => {
  const auth = inject(AuthService);
  return auth.jeton()
    ? true
    : inject(Router).createUrlTree(['/connexion'], { queryParams: { retour: state.url } });
};

/** Routes réservées aux administrateurs. */
export const adminGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  return auth.estAdmin() ? true : inject(Router).createUrlTree(['/']);
};

/** Pages de connexion et d'inscription : inutiles si on est déjà connecté. */
export const invitesGuard: CanActivateFn = () => {
```

```
  const auth = inject(AuthService);
  return auth.jeton() ? inject(Router).createUrlTree(['/']) : true;
};
```

13. Frontend Angular : les pages

Page	Route	Ce qu'elle montre
Connexion	`/connexion`	Formulaire réactif, boutons de comptes de démo, message 401 clair, retour à la page demandée
Inscription	`/inscription`	Validateur de groupe (mots de passe identiques), erreurs du serveur par champ
Tableau de bord	`/`	Cartes de statistiques, recherche avec debounce, filtres, tri et pagination côté serveur, suppression (ADMIN)
Détail	`/tickets/:id`	Changement de statut filtré selon le rôle, assignation (ADMIN), boutons selon les droits
Formulaire	`/tickets/nouveau`, `/tickets/:id/modifier`	Même composant en création et en modification
Utilisateurs	`/utilisateurs`	Changement de rôle, protégé par `adminGuard`

L'affichage selon le rôle

Le rôle est exposé par le signal calculé `auth.estAdmin()`. Les templates s'en servent pour afficher ou masquer des éléments, et comme c'est un signal, l'interface se met à jour seule à la connexion ou à la déconnexion.

```
<!-- app.html : lien de menu réservé aux administrateurs -->
@if (auth.estAdmin()) {
  <a mat-button routerLink="/utilisateurs">Utilisateurs</a>
}

// tableau-de-bord.ts : la colonne "actions" (corbeille) n'existe que pour un ADMIN
protected readonly colonnes = computed(() => {
  const base = ['id', 'titre', 'type', 'severite', 'statut', 'assigne', 'creeLe'];
  return this.auth.estAdmin() ? [...base, 'actions'] : base;
});
```

Le tableau de bord réactif

```
  // ---- Critères de recherche (signals) ----
  // Champ de recherche : on attend 300 ms après la dernière frappe (debounceTime) avant de chercher,
  // pour ne pas envoyer une requête à chaque lettre. toSignal() convertit l'Observable en signal.
  protected readonly rechercheCtrl = new FormControl('', { nonNullable: true });
  private readonly recherche = toSignal(
    this.rechercheCtrl.valueChanges.pipe(
      debounceTime(300),
      map((v) => v.trim()),
      distinctUntilChanged(),
    ),
    { initialValue: '' },
  );
  protected readonly statut = signal<Statut | null>(null);
  protected readonly severite = signal<Severite | null>(null);
  protected readonly type = signal<TypeTicket | null>(null);
  protected readonly assignesAMoi = signal(false);
  protected readonly pageIndex = signal(0);
  protected readonly pageTaille = signal(10);
  protected readonly tri = signal('creeLe,desc');
  private readonly rafraichissement = signal(0);

  private readonly filtre = computed<FiltreTickets>(() => ({
```

```
  recherche: this.recherche(),
  statut: this.statut(),
  severite: this.severite(),
  type: this.type(),
  assigneId: this.assignesAMoi() ? (this.auth.utilisateur()?.id ?? null) : null,
}));

// ---- État affiché ----
protected readonly lignes = signal<Ticket[]>([]);
protected readonly total = signal(0);
protected readonly chargement = signal(false);
protected readonly erreur = signal<string | null>(null);
protected readonly stats = signal<Statistiques | null>(null);

constructor() {
  // Revenir à la première page dès qu'un filtre change
  effect(() => {
    this.filtre();
    this.pageIndex.set(0);
  });

  // effect() : s'exécute une première fois, puis à chaque changement d'un signal lu à l'intérieur.
  // Recharger dès qu'un critère change (filtre, page, tri ou rafraîchissement manuel)
  effect((nettoyage) => {
    const filtre = this.filtre();
    const page = this.pageIndex();
    const taille = this.pageTaille();
    const tri = this.tri();
    this.rafraichissement();

    this.chargement.set(true);
    const abonnement = this.tickets.rechercher(filtre, page, taille, tri).subscribe({
      next: (resultat) => {
        this.lignes.set(resultat.content);
        this.total.set(resultat.page.totalElements);
        this.erreur.set(null);
        this.chargement.set(false);
      },
      error: (err) => {
        this.erreur.set(messageErreur(err));
        this.chargement.set(false);
      },
    });
    nettoyage(() => abonnement.unsubscribe()); // annule la requête précédente si un critère change
  });

  this.chargerStatistiques();
}

// Les cellules de mat-table sont typées "any" : ces méthodes rétablissent le typage
protected libelleType(t: Ticket): string {
  return LIBELLES_TYPE[t.type];
}

protected libelleSeverite(t: Ticket): string {
  return LIBELLES_SEVERITE[t.severite];
}

protected libelleStatut(t: Ticket): string {
  return LIBELLES_STATUT[t.statut];
}
```

Les tests du frontend

Fichier	Ce qu'il vérifie
`auth.service.spec.ts`	Session enregistrée au login, restaurée au rafraîchissement, ignorée si expirée
`auth.interceptor.spec.ts`	Header Authorization ajouté ; session fermée sur un 401
`auth.guards.spec.ts`	Redirection vers `/connexion?retour=...` ; `adminGuard` refuse un USER
`ticket.service.spec.ts`	Paramètres de recherche : filtres vides ignorés, pagination et tri
`tableau-de-bord.spec.ts`	Affichage des tickets ; bouton Supprimer visible seulement pour un ADMIN
`connexion.spec.ts`	Pas d'appel API si le formulaire est invalide ; message clair sur un 401
`ticket-formulaire.spec.ts`	Validateur `nonVide` (équivalent de `@NotBlank`)

Les tests utilisent `HttpTestingController` : aucune requête réelle ne part, le test décide de la réponse (`flush`) et vérifie la requête envoyée (`expectOne`).

14. Docker

Chaque partie a un Dockerfile **multi-stage** : une première image (lourde) compile le projet, une seconde (légère) ne contient que le résultat. L'image finale ne contient ni Maven, ni Node, ni le code source.

Backend

backend/Dockerfile

```
# ---------- Étape 1 : build avec Maven ----------
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
# Les dépendances d'abord : cette couche reste en cache tant que pom.xml ne change pas
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
RUN mvn -B -q package -DskipTests

# ---------- Étape 2 : image d'exécution légère ----------
FROM eclipse-temurin:21-jre
WORKDIR /app
# Ne jamais exécuter l'application en root dans un conteneur
RUN groupadd --system secutrack && useradd --system --gid secutrack secutrack
COPY --from=build /app/target/*.jar app.jar
USER secutrack
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

Frontend et nginx

frontend/Dockerfile

```
# ---------- Étape 1 : build Angular ----------
FROM node:24-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npx ng build

# ---------- Étape 2 : nginx sert les fichiers statiques ----------
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist/frontend/browser /usr/share/nginx/html
EXPOSE 80
```

frontend/nginx.conf

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;
    server_tokens off;                      # ne pas afficher la version de nginx

    # En-têtes de sécurité de base
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "no-referrer" always;

    # Les appels /api sont relayés vers le backend : même origine pour le navigateur, donc pas de CORS
    location /api/ {
        proxy_pass http://backend:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Swagger UI du backend, accessible via le même port
    location ~ ^/(swagger-ui|v3/api-docs) {
        proxy_pass http://backend:8080;
    }

    # Fichiers avec hash dans le nom : cache long
    location ~* \.(js|css|woff2|ico)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Routage Angular : toute URL inconnue renvoie index.html
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

nginx sert les fichiers Angular et relaie `/api` vers le conteneur `backend`. Pour le navigateur, l'application et l'API ont la même origine (http://localhost:8081) : aucune configuration CORS n'est nécessaire en production.

docker-compose.yml

docker-compose.yml

```
# SecuTrack complet : PostgreSQL + API Spring Boot + frontend Angular (nginx)
# Lancement : docker compose up --build   puis   http://localhost:8081
services:
  # Base de données PostgreSQL. Les données sont conservées dans le volume "pgdata".
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: secutrack
      POSTGRES_USER: secutrack
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-secutrack}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    # healthcheck : Docker vérifie que PostgreSQL accepte les connexions avant de démarrer le backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U secutrack -d secutrack"]
      interval: 5s
      timeout: 5s
      retries: 10

  # API Spring Boot, construite avec backend/Dockerfile
  backend:
    build: ./backend
    environment:
      SPRING_PROFILES_ACTIVE: docker          # charge application-docker.yml (PostgreSQL)
      DB_PASSWORD: ${POSTGRES_PASSWORD:-secutrack}
      JWT_SECRET: ${JWT_SECRET:-secret-docker-a-remplacer-par-une-valeur-aleatoire-longue}
      DEMO_ACTIF: ${DEMO_ACTIF:-true}
    ports:
      - "8080:8080"
    depends_on:
      db:
        condition: service_healthy            # attend que la base soit prête

  # Angular servi par nginx, qui relaie aussi /api vers le service "backend"
  frontend:
    build: ./frontend
    ports:
      - "8081:80"
    depends_on:
```

```
      - backend

volumes:
  pgdata:
```

Commande	Effet
`docker compose up --build`	Construit les images et démarre tout
`docker compose up -d`	Démarre en arrière-plan
`docker compose logs -f backend`	Suit les journaux de l'API
`docker compose ps`	État des conteneurs
`docker compose down`	Arrête et supprime les conteneurs (les données restent dans le volume)
`docker compose down -v`	Supprime aussi le volume PostgreSQL : base remise à zéro

15. CI/CD avec GitHub Actions

Le workflow s'exécute à chaque push et à chaque pull request. Les jobs `backend` et `frontend` tournent en parallèle ; le job `images` attend qu'ils réussissent. Sur `main` seulement, les images sont publiées dans GitHub Container Registry : c'est la partie « CD » (livraison continue).

`.github/workflows/ci-cd.yml`

```
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read
  packages: write        # pour publier les images dans GitHub Container Registry

jobs:
  backend:
    name: Backend (Maven)
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
          cache-dependency-path: backend/pom.xml

      - name: Build et tests
        run: ./mvnw -B verify

      - name: Rapports de tests (en cas d'échec)
        if: failure()
        uses: actions/upload-artifact@v4
        with:
          name: rapports-surefire
          path: backend/target/surefire-reports/

  frontend:
    name: Frontend (Angular)
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '24'
          cache: npm
          cache-dependency-path: frontend/package-lock.json

      - run: npm ci
      - name: Tests
        run: npx ng test --watch=false
      - name: Build
        run: npx ng build

  images:
    name: Images Docker
```

```
    runs-on: ubuntu-latest
    needs: [backend, frontend]
    # Publication seulement sur main ; sur une pull request, on vérifie juste que les images se construisent
    env:
      PUBLIER: ${{ github.event_name == 'push' && github.ref == 'refs/heads/main' }}
    strategy:
      matrix:
        include:
          - nom: secutrack-api
            contexte: backend
          - nom: secutrack-web
            contexte: frontend
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3

      - name: Connexion à GHCR
        if: env.PUBLIER == 'true'
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Tags de l'image
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ghcr.io/${{ github.repository_owner }}/${{ matrix.nom }}
          tags: |
            type=sha
            type=raw,value=latest,enable={{is_default_branch}}

      - name: Build (et push sur main)
        uses: docker/build-push-action@v6
        with:
          context: ${{ matrix.contexte }}
          push: ${{ env.PUBLIER == 'true' }}
          tags: ${{ steps.meta.outputs.tags }}
          cache-from: type=gha,scope=${{ matrix.nom }}
          cache-to: type=gha,mode=max,scope=${{ matrix.nom }}
```

Dependabot

`.github/dependabot.yml` ouvre automatiquement des pull requests de mise à jour des dépendances Maven, npm, des actions GitHub et des images Docker. Chaque PR passe par la CI : si les tests restent verts, la mise à jour est sûre. C'est une forme de vigie technologique automatisée.

Protéger la branche main

Dans GitHub, **Settings → Branches → Add rule** sur `main` : cocher « Require a pull request before merging » et « Require status checks to pass » (jobs Backend et Frontend). Plus personne ne peut pousser directement sur `main` : tout passe par une PR testée, comme en entreprise.

16. TeamCity

TeamCity est le serveur CI de JetBrains. Il a une architecture **serveur + agents** : le serveur orchestre et affiche les résultats, les agents exécutent les builds. Le dossier `teamcity/` lance les deux en local avec Docker ; l'agent est préparé avec Java 21 et Node 24.

Mise en route

```
cd teamcity
docker compose up -d --build
```

Étape	Action
1	Ouvrir http://localhost:8111, choisir la base interne, accepter la licence, créer le compte admin
2	**Agents → Unauthorized** : autoriser `secutrack-agent`
3	**Projects → Create project → From a repository URL** : coller l'URL GitHub du dépôt
4	Choisir **Import settings from .teamcity/settings.kts** : les builds Backend et Frontend apparaissent
5	Pousser un commit : TeamCity déclenche les deux builds

Le pipeline en code (Kotlin DSL)

`.teamcity/settings.kts`

```
versionné avec le code (Kotlin DSL).
 * TeamCity lit ce fichier quand les « Versioned Settings » du projet sont activés (voir README).
 */

version = "2025.11"

project {
    description = "SecuTrack : API Spring Boot + frontend Angular"

    buildType(Backend)
    buildType(Frontend)

    buildTypesOrder = arrayListOf(Backend, Frontend)
}

object Backend : BuildType({
    name = "Backend - build et tests Maven"
    description = "Compile l'API, exécute les tests JUnit et produit le jar"

    artifactRules = "backend/target/*.jar"

    vcs {
        root(DslContext.settingsRoot)
    }

    steps {
        script {
            name = "mvnw verify"
            workingDir = "backend"
            scriptContent = """
                export JAVA_HOME=${'$'}JDK21
                ./mvnw -B verify
            """.trimIndent()
        }
    }

    triggers {
```

```
        vcs { }        // un build à chaque push
    }

    features {
        xmlReport {    // résultats JUnit visibles dans l'onglet Tests
            reportType = XmlReport.XmlReportType.JUNIT
            rules = "backend/target/surefire-reports/*.xml"
        }
    }
})

object Frontend : BuildType({
    name = "Frontend - tests et build Angular"
    description = "npm ci, tests Vitest, build de production"

    artifactRules = "frontend/dist/frontend/browser/** => secutrack-web.zip"

    vcs {
        root(DslContext.settingsRoot)
    }

    steps {
        script {
            name = "npm ci, ng test, ng build"
            workingDir = "frontend"
            scriptContent = """
                npm ci
                npx ng test --watch=false
                npx ng build
            """.trimIndent()
        }
    }

    triggers {
        vcs { }
    }
})
```

TeamCity et GitHub Actions

	GitHub Actions	TeamCity
Configuration	YAML dans `.github/workflows/`	Kotlin DSL dans `.teamcity/` ou interface web
Exécution	Runners hébergés par GitHub (ou auto-hébergés)	Agents installés par l'organisation
Déclencheurs	`on: push`, `pull_request`…	VCS triggers, planification, dépendances entre builds
Points forts	Intégré au dépôt, énorme catalogue d'actions	Chaînes de builds, historique, contrôle fin des agents

17. Contribuer : commits et pull requests

Le fichier `CONTRIBUTING.md` à la racine du projet décrit la façon de travailler en équipe. En résumé : **un ticket → une branche → des commits propres → une pull request → une revue → un merge**.

Le cycle d'un ticket

```
git switch main && git pull                          # 1. partir de la dernière version
git switch -c feature/12-date-echeance               # 2. une branche par ticket (numéro de l'issue)
./mvnw verify ; npx ng test --watch=false            # 3. tester
git add <fichiers>
git commit -m "feat(ticket): ajoute la date d'échéance"
git push -u origin feature/12-date-echeance          # 4. pousser
gh pr create --fill                                  # 5. ouvrir la PR (ou via le site GitHub)
```

Nommer les branches

Préfixe	Usage	Exemple
`feature/`	Nouvelle fonctionnalité	`feature/12-date-echeance`
`fix/`	Correction d'anomalie	`fix/18-tri-inconnu-500`
`test/`	Tests	`test/21-testcontainers`
`docs/`	Documentation	`docs/connexion-postgresql`
`refactor/`	Restructuration sans changement de comportement	`refactor/ticket-mapper`
`chore/`	Maintenance (dépendances, configuration, CI)	`chore/spring-boot-4`

Écrire de bons commits : Conventional Commits

Format : `type(portée): description courte au présent`. Un commit correspond à **un seul changement logique**, avec une description de moins de 72 caractères. Un corps facultatif, après une ligne vide, explique le *pourquoi*. La mention `Closes #24` ferme automatiquement l'issue 24 au merge.

Type	Quand	Exemple
`feat`	Nouvelle fonctionnalité	`feat(ticket): ajoute le filtre « créés par moi »`
`fix`	Correction	`fix(auth): renvoie 401 si le courriel est vide`
`test`	Tests	`test(ticket): vérifie qu'un USER ne peut pas supprimer`
`docs`	Documentation	`docs: explique la connexion avec DBeaver`
`refactor`	Restructuration	`refactor(ticket): extrait la vérification des droits`
`style`	Formatage	`style(frontend): applique prettier`
`chore`	Maintenance	`chore(deps): met à jour Angular Material`
`ci`	Pipelines	`ci: publie les images seulement sur main`

```
fix(ticket): bloque la modification d'un ticket fermé

Un ticket fermé pouvait encore être réassigné par un ADMIN.
La vérification est maintenant faite aussi dans assigner().

Closes #24
```

Attention — Jamais de secret dans un commit (mot de passe, jeton, fichier `.env`). Un secret poussé sur GitHub doit être considéré comme compromis : on le change immédiatement, puis on le retire de l'historique.

La pull request

La PR ne se crée pas dans le projet mais **sur GitHub**. Après le `git push`, un bandeau « Compare & pull request » apparaît sur la page du dépôt. Le formulaire est pré-rempli par `.github/pull_request_template.md`. Ensuite :

Étape	Ce qui se passe
1	GitHub Actions lance les tests backend, frontend et la construction des images
2	TeamCity, s'il est configuré, lance aussi ses builds
3	Un relecteur commente ; pour corriger, nouveaux commits sur la même branche puis `git push`
4	CI verte et PR approuvée : **Squash and merge**, puis suppression de la branche

```
gh pr create --title "feat(ticket): date d'échéance" --body "Closes #12"
gh pr checks                              # état de la CI
gh pr view --web                          # ouvrir dans le navigateur
gh pr merge --squash --delete-branch      # merger quand tout est vert
```

Faire une revue de code

Ordre	Question à se poser
1. Le besoin	Le code respecte-t-il les critères d'acceptation du ticket ?
2. La sécurité	Droits vérifiés dans le backend ? Données sensibles exposées dans un DTO, un log, une erreur ?
3. Les tests	Le nouveau comportement est-il testé, y compris les cas 400, 403, 404, 409 ?
4. La lisibilité	Noms clairs, méthodes courtes, commentaires utiles ?
5. Les détails	Formatage, conventions de l'équipe

Formule tes remarques comme des questions : « Que se passe-t-il si `assigneId` est null ici ? » plutôt que « C'est faux ».

Les modèles d'issues

Le dossier `.github/ISSUE_TEMPLATE/` contient deux formulaires : **User story** (rôle, besoin, bénéfice, critères d'acceptation, priorité, estimation) et **Anomalie** (étapes de reproduction, résultat attendu et obtenu, gravité). Sur GitHub, le bouton « New issue » propose directement ces deux modèles.

18. Analyse des besoins et Agile

Le poste inclut l'analyse des besoins des utilisateurs, le rôle d'analyste fonctionnel au besoin et le travail en environnement Agile. Le document `docs/ANALYSE-ET-AGILE.md` détaille cette partie.

Du besoin à la user story

Le besoin exprimé : « Les signalements arrivent par courriel, on en perd, on ne sait pas qui s'en occupe, on n'a aucune vue d'ensemble. » L'analyse le découpe en trois problèmes, chacun couvert par une fonctionnalité : création de tickets typés, assignation et statuts, tableau de bord et statistiques.

Une **user story** s'écrit : *En tant que <rôle>, je veux <action>, afin de <bénéfice>*. Ses **critères d'acceptation**, au format Étant donné / Quand / Alors, disent quand elle est terminée et deviennent des tests.

```
Fonctionnalité: Résolution des tickets réservée aux administrateurs

  Scénario: Un utilisateur tente de résoudre un ticket
    Étant donné que je suis connecté en tant que USER
    Et que le ticket 7 est EN_COURS
    Quand je demande à passer le ticket 7 à RESOLU
    Alors je reçois une réponse 403
    Et le ticket 7 reste EN_COURS
```

Ce scénario correspond exactement au test `TicketApiIntegrationTest.seulLAdminPeutResoudre` : chaque ligne y a son équivalent dans le code.

Les user stories réalisées

#	User story	Preuve
US-1	Employé : créer un compte pour déclarer des tickets	`inscriptionPuisCreationDeTicket`
US-2	Employé : déclarer un ticket pour qu'il soit traité	`TicketServiceTest.creer`
US-3	Analyste : filtrer et chercher les tickets	`ticket.service.spec.ts`
US-4	Administrateur : assigner un ticket	`@PreAuthorize` + test Postman 403
US-5	Administrateur : être le seul à résoudre ou fermer	`seulLAdminPeutResoudre`
US-6	Administrateur : verrouiller un ticket fermé	`ticketFermeBloque`
US-7	Responsable : voir la situation d'un coup d'œil	`tableau-de-bord.spec.ts`

Organiser un sprint avec GitHub Projects

Cérémonie	Sur SecuTrack	Durée
Planification	Choisir 2 ou 3 stories (≈ 8 points), les passer dans la colonne Sprint	15 min
Mêlée quotidienne	Hier, aujourd'hui, blocages	5 min
Revue de sprint	Démontrer les stories terminées	15 min
Rétrospective	Ce qui a marché, ce qui est à améliorer	10 min

Définition de « terminé » : critères d'acceptation vérifiés, tests ajoutés, CI verte (GitHub Actions et TeamCity), code commenté et documentation à jour, PR relue et mergée.

La démarche d'analyste fonctionnel

Avant de coder une demande : reformuler le besoin et chercher le vrai problème (« pourquoi ? ») ; identifier les acteurs et leurs droits ; écrire les stories et leurs critères et les faire valider ; repérer les impacts (données, API, écrans, sécurité, performance) ; découper en stories livrables en quelques jours.

19. Tester l'API et explorer la base

Postman

Le dossier `docs/postman/` contient une collection prête à importer, avec des **tests automatiques** sur chaque requête, et un environnement pour l'application lancée avec Docker. Le guide complet est dans `docs/TESTER-L-API.md`.

Dossier de la collection	Contenu
1. Authentification	Login ADMIN (le jeton est enregistré automatiquement dans `{{token}}`), inscription, profil
2. Tickets	Recherche, statistiques, création (l'id est mémorisé dans `{{ticketId}}`), modification, statut, assignation
3. Cas d'erreur	401, 400, 404, 409, puis bascule en USER pour obtenir un 403
4. Nettoyage	Suppression du ticket de test

```
// Script "Tests" de la requête Login ADMIN : enregistre le jeton pour toute la collection
pm.test('Statut 200', () => pm.response.to.have.status(200));
const corps = pm.response.json();
pm.collectionVariables.set('token', corps.token);
pm.test('Rôle ADMIN', () => pm.expect(corps.utilisateur.role).to.eql('ADMIN'));
```

Pour tout lancer d'un coup : bouton **Run** de la collection (Collection Runner). C'est une forme de test d'assurance qualité de l'API, complémentaire des tests JUnit.

La base de données

Paramètre	Valeur
URL JDBC	`jdbc:postgresql://localhost:5432/secutrack`
Utilisateur / mot de passe	`secutrack` / `secutrack`
Clients compatibles	DBeaver, pgAdmin, IntelliJ (Database), SQL Workbench/J (avec le pilote PostgreSQL)
Non compatible	MySQL Workbench (MySQL et MariaDB seulement)
Sans rien installer	`docker compose exec db psql -U secutrack -d secutrack`

Le fichier `docs/sql/requetes-utiles.sql` contient des requêtes commentées : jointures tickets / utilisateurs, statistiques, index, plan d'exécution (`EXPLAIN ANALYZE`) et observation du verrouillage optimiste.

```
-- 3. Les tickets avec le nom du créateur et de l'assigné (jointures)
SELECT t.id, t.titre, t.type, t.severite, t.statut,
       c.nom_complet AS createur,
       a.nom_complet AS assigne,
       t.cree_le, t.version
FROM tickets t
JOIN utilisateurs c      ON c.id = t.createur_id
LEFT JOIN utilisateurs a ON a.id = t.assigne_id      -- LEFT : garde les tickets non assignés
ORDER BY t.cree_le DESC;
```

20. Les outils d'IA dans le développement

Le poste demande une connaissance des outils d'IA appliqués au développement. Le document `docs/IA-DEVELOPPEMENT.md` propose sept exercices sur SecuTrack.

Les règles d'or

Règle	Pourquoi
Aucune donnée sensible dans un outil non approuvé	Secrets, données clients et code interne ne sortent pas de l'organisation
Tu restes responsable du code	Si tu ne peux pas l'expliquer en revue, tu ne le commites pas
Les tests sont l'arbitre	Un code généré n'est accepté que si la CI est verte
Relire la sécurité en priorité	Requêtes concaténées, droits vérifiés seulement côté client, CSRF désactivé sans raison
Vérifier les versions	Les outils mélangent souvent `javax`/`jakarta` ou proposent des API supprimées

Les exercices

#	Tâche	Vérification
IA-1	Expliquer `TicketSpecifications` et le SQL généré	Comparer avec `show-sql: true`
IA-2	Générer les tests de `UtilisateurService.changerRole`	Casser la règle : un test doit échouer
IA-3	Revue de sécurité de `SecuriteConfig` et `TicketService`	Chaque remarque est-elle fondée ?
IA-4	Implémenter SEC-2 (backend + frontend + tests)	Lire tout le diff avant de committer
IA-5	Rédiger la documentation d'une fonctionnalité	Exactitude par rapport au code
IA-6	Diagnostiquer une erreur de build	Comprendre la cause, pas seulement le correctif
IA-7	Lister les changements pour Spring Boot 4.1	Comparer avec le guide officiel

> **Bonne pratique** — Le « test de mutation manuel » de l'exercice IA-2 est une excellente habitude : supprimer temporairement une règle métier et vérifier qu'au moins un test échoue. Sinon, les tests ne protègent rien, qu'ils aient été écrits par toi ou par une IA.

21. Vigie technologique et exploitation

La vigie

La vigie consiste à savoir quelles versions on utilise, quand elles ne seront plus supportées, et à proposer les mises à jour au bon moment (`docs/VIGIE.md`).

Composant	Version	Situation (octobre 2026)	Action
Spring Boot	3.5.16	Support open source terminé le 30 juin 2026	Migrer vers 4.1 (SEC-10)
Java	21 LTS	Supporté	Rester sur 21
Angular	22	Version actuelle	`ng update` régulier
PostgreSQL	16	Supporté	—
Node.js	24 LTS	Exigé par Angular 22	—

Outils : Dependabot (PR de mises à jour automatiques), `./mvnw versions:display-dependency-updates`, `npm outdated`, `npm audit`, endoflife.date et les blogs officiels. Pour influencer une décision, une note courte (constat, risque, options, recommandation, preuve) vaut mieux qu'un long rapport.

L'exploitation

Le document `docs/EXPLOITATION.md` couvre la documentation « d'exploitation et d'entretien » demandée par le poste : composants, variables d'environnement, démarrage et mise à jour, surveillance, sauvegarde, incidents courants et rotation du secret JWT.

Vérification	Commande	Attendu
Santé de l'API	`curl localhost:8080/actuator/health`	`{"status":"UP"}`
Journaux	`docker compose logs -f backend`	Aucune ligne `ERROR`
Sauvegarde	`docker compose exec -T db pg_dump -U secutrack secutrack > sauvegarde.sql`	Fichier SQL

Le guide utilisateur

`docs/GUIDE-UTILISATEUR.md` s'adresse aux utilisateurs finaux : connexion, tableau de bord, déclaration et suivi d'un ticket, droits par rôle, questions fréquentes. Il sert aussi de modèle de matériel de formation, autre activité mentionnée dans la description du poste.

22. Couverture du poste et plan de mise à niveau

Ce que le projet couvre

Exigence ou activité du poste	Dans SecuTrack
Java et Spring Boot pour le backend web	API complète, sécurité, JPA, validation, erreurs
Angular, Node et bibliothèques open source	Angular 22, Angular Material, RxJS ; Node pour le build et les tests
Docker	Dockerfiles multi-stage, docker compose, nginx
CI/CD TeamCity, GitHub, GitHub Actions	Workflow CI/CD + GHCR, pipeline TeamCity en Kotlin DSL
Outils d'IA en développement	`IA-DEVELOPPEMENT.md`, 7 exercices
Analyser les besoins, analyste fonctionnel	User stories, critères d'acceptation, exercice d'atelier
Environnement Agile	Backlog, GitHub Projects, Definition of Done, sprints
Tests unitaires et d'assurance qualité	JUnit, Mockito, MockMvc, Vitest, collection Postman
Vigie de l'évolution des logiciels	`VIGIE.md`, Dependabot, ticket de migration
Documentation (développement, exploitation, entretien)	Code commenté, ce PDF, `EXPLOITATION.md`
Assister les utilisateurs, matériel de formation	`GUIDE-UTILISATEUR.md`
Collaboration avec la cybersécurité	Domaine de l'application, choix de sécurité documentés

> **Contexte Hydro-Québec** — Le projet couvre la technique et les pratiques. Il ne remplace pas la connaissance du contexte réel : l'application de l'équipe, ses versions, son pipeline et ses conventions. Les questions de la partie 5 de `EXERCICES.md` servent à les découvrir dès la première semaine.

Le plan de mise à niveau

Étape	Contenu
1	Lancer avec Docker et explorer l'application avec les comptes de démo
2	Pousser sur GitHub, protéger `main`, faire passer la CI
3	Lire le code commenté (backend puis frontend) avec ce PDF à côté
4	Tester l'API dans Postman et explorer la base dans un client SQL
5	Exercices Git G1 à G8, puis TeamCity T1 à T4
6	Créer le tableau GitHub Projects et les issues SEC-1 à SEC-10
7	Sprint 1 : SEC-1, SEC-2, SEC-7
8	Sprint 2 : SEC-3 (avec l'exercice d'analyse), SEC-4
9	Exercices IA-1 à IA-3
10	Sprint 3 : SEC-5, SEC-6, note de vigie et migration SEC-10

23. Aide-mémoire des annotations

Spring et Spring Boot

Annotation	Rôle	Où
`@SpringBootApplication`	Configuration + auto-configuration + scan	`SecutrackApplication`
`@Configuration` / `@Bean`	Déclarer des beans	`config/`
`@ConfigurationProperties`	Lier un bloc du YAML à un record	`SecutrackProperties`
`@ConditionalOnProperty`	Bean créé seulement si une propriété vaut X	`DonneesDemo`
`@Component` / `@Service`	Bean générique / métier	`TicketMapper` / `TicketService`
`@RestController` / `@RequestMapping`	Contrôleur JSON / préfixe d'URL	`*Controller`
`@Get/Post/Put/Patch/DeleteMapping`	Verbe HTTP + chemin	`TicketController`
`@PathVariable` / `@RequestBody`	Valeur de l'URL / corps JSON	Contrôleurs
`@ResponseStatus`	Code HTTP fixe	`register` (201), `supprimer` (204)
`@PageableDefault` / `@ParameterObject`	Pagination par défaut / documentation Swagger	`rechercher`
`@Valid`	Déclenche la validation	Devant `@RequestBody`
`@NotBlank` `@NotNull` `@Size` `@Email` `@Pattern` `@Positive`	Contraintes	DTO de requête
`@RestControllerAdvice` / `@ExceptionHandler`	Gestion globale des erreurs	`GlobalExceptionHandler`
`@Transactional`	Transaction (commit / rollback)	Services
`@EnableWebSecurity` / `@EnableMethodSecurity`	Active la sécurité / `@PreAuthorize`	`SecuriteConfig`
`@PreAuthorize("hasRole('ADMIN')")`	Autorisation sur une méthode	Assigner, supprimer, utilisateurs
`@AuthenticationPrincipal`	Injecte le JWT de l'utilisateur connecté	Contrôleurs
`@Tag` / `@Operation`	Documentation OpenAPI	Contrôleurs

JPA et Spring Data

Annotation	Rôle
`@Entity` / `@Table`	Classe ↔ table
`@Id` / `@GeneratedValue`	Clé primaire générée
`@Column`	Contraintes de colonne

Annotation	Rôle
`@Enumerated(EnumType.STRING)`	Enum en texte
`@ManyToOne` / `@JoinColumn`	Clé étrangère
`@Version`	Verrouillage optimiste
`@PrePersist` / `@PreUpdate`	Code avant INSERT / UPDATE
`@Query` / `@Param`	Requête JPQL et ses paramètres
`@EntityGraph`	Charge des relations dans la même requête (anti N+1)

Tests

Annotation	Rôle
`@Test` / `@DisplayName` / `@BeforeEach`	JUnit 5
`@ExtendWith(MockitoExtension.class)`	Mockito sans Spring
`@Mock` / `@Spy` / `@InjectMocks`	Doublures de test
`@SpringBootTest` / `@AutoConfigureMockMvc`	Application complète + MockMvc

24. Dépannage

Symptôme	Cause probable et solution
`./mvnw: Permission denied`	`chmod +x backend/mvnw` (ou `git update-index --chmod=+x backend/mvnw`)
`secutrack.jwt.secret doit contenir au moins 32 caractères`	Variable `JWT_SECRET` trop courte
Le frontend affiche « Impossible de joindre l'API »	Le backend n'est pas démarré, ou n'écoute pas sur 8080
Toutes les requêtes renvoient 401	Jeton expiré (120 min) : se reconnecter. Ou secret JWT changé entre deux démarrages
403 inattendu	Rôle insuffisant : vérifier le rôle dans le jeton (jwt.io) et la règle concernée
`LazyInitializationException`	Relation LAZY lue hors transaction : convertir en DTO dans le service
Données de démo absentes avec Docker	La base n'était pas vide (volume existant) : `docker compose down -v`
Port 5432, 8080 ou 8081 déjà utilisé	Arrêter l'autre programme ou changer le port dans `docker-compose.yml`
`ng` : version de Node non supportée	Installer Node.js 24 LTS
Erreur CORS dans le navigateur	Origine absente de `secutrack.cors-origines` (développement sans proxy)